PROTEL

Procedure Oriented Type Enforcing Language

Language Reference Manual

Version 2026

Derived from the February 1989 revision

Resurrected and enhanced for the 21st century

by

Vincente D'Ingianni
vincente@binary-systems.com

former Advisor and Manager,
Bell-Northern Research / Nortel DMS-500 Development

Table of Contents

Preface

From 1988 through 1998, I had the privilege of developing cutting edge real time software on state of the art communication systems. I regularly sifted through millions of lines of code with extremely primitive tools by today's standards. As the team leader for Carrier AIN, we had claim to the first feature delivered to the field using the object oriented extensions in PROTEL 2.

When I taught the Introduction to PROTEL classes at BNR/Nortel in the early 1990s, I would often get questions from a new hires asking, "Why don't we use C and Unix?".

Besides the fact that C and Unix were created at our competitor AT&T, there were many excellent features of PROTEL that made it superior to C. These features protected developers from stupid mistakes and enforced good programming structure and readability without losing efficiency.

Modern programming languages in the 21st century have lost their way with overuse of classes and inheritance, inefficient code, and ridiculous overhead requirements. This is evident in the resurgence of procedural programming over object oriented programming methodology. Strong typing aspects allow errors to be caught at compile time, not run time.

Its all in the name, Procedure Oriented Type Enforcing Language.

This is not retro computing, it is smart computing.

This manual was recreated to resurrect a fantastic language that was left for dead through bankruptcy and corporate mismanagement. Through in-depth web searches, foggy memory, and a lot of Artificial Intelligence, this new PROTEL Reference Manual will give the next generation of software engineers the opportunity to experience a programming language that was ahead of its time.

This new PROTEL Reference Manual is dedicated to the BNR Compiler Team and all of the Nortel developers that produced over 50 million lines of PROTEL code that have been hidden from the public eye. Much of this code is still in use in the 21st century. They created the modern communications network that we enjoy today, but get very little public recognition. These guys did the heavy lifting with the creation and maturation of this programming language.

Finally, this PROTEL Reference Manual and the PROTEL Introductory Manual are the foundation of a new public domain compiler implementation that will finally give those new hires what they needed — a modern version of PROTEL that fully integrates with C/C++ and UNIX / Linux operating systems.

Key updates in PROTEL 2026 include;

- Case sensitivity by default for identifiers (types, variables, procedures, etc.), with keywords required in UPPERCASE
- Backward compatibility via compiler option --classical for case-insensitivity (matching original Nortel DMS behavior)

- Modern compilation using GCC backend for Mac OS X and UNIX / Linux, with interoperability for calling/linking C code

This manual is structured similarly to the original OFLPR for familiarity, with additions for object oriented extensions and modern features.

1.0 Introduction

The primary objective of the PROTEL language is to facilitate the implementation of reliable and maintainable software for stored program switching systems. The most significant feature of the language in support of reliability is the incorporation of user-defined data types which allows extensive type checking at compile time. In telephone switching applications, the complex data structures required are fully supported through the inclusion of data aggregates and pointer variables. The unreliability inherent in the use of pointers is minimized by constraining them to point to objects of one specific type. The clarity and, to a large extent, the reliability of programs written in PROTEL is enhanced by its single entry/exit control structures. Combining good structured programming techniques with these control structures has been shown to yield programs which are easier to understand and modify.

This reference manual contains a formal description of the PROTEL syntax together with an informal presentation of its semantics.

In early 1990s, there were PROTEL implementations for three computers: the NT40 (PROTEL/NT40), the Motorola 68020 (PROTEL/68K) and the IBM/370 (PROTEL/370). The first two are known generically as PROTEL/DMS, or the "DMS implementation." This follows from the design goal of PROTEL/68K to preserve the semantics of PROTEL/NT40. The IBM/370 implementation did not have this design goal.

Shortly thereafter, PROTEL/88K for the Motorola 88000 series RISC processors was introduced. This was followed by PROTEL on the PowerPC line of processors in the XA-Core DMS systems.

PROTEL 2026 is a modern strongly typed, block-structured language designed for real-time applications, particularly telecommunications. It enforces type safety at compile-time, supports modular programming, and includes object-oriented extensions from PROTEL-2.
The language continues to promote reliable, maintainable code through:

- Strict type checking and compatibility rules.
- Modular sections (interfaces and implementations).
- Single-entry/single-exit control flow.
- Aggregates like STRUCT, TABLE, DESC, AREA, and OO CLASS.

PROTEL 2026 compiles to native executables or shared libraries using GCC, allowing seamless integration with C (e.g., PROTEL PROCs callable as C functions via extern "C").

2.0 Notation

This section presents the notation used throughout the manual to describe PROTEL. The syntax of the language is described using the BNF (Backus-Naur form) notation. The special symbols (metasymbols) of BNF are shown below. Any symbol or string of characters not defined below is called a terminal symbol in the sense that it represents itself. A terminal symbol consisting of a string of upper case characters is considered to be a single indivisible symbol and must appear in the language without embedded blanks.

`< >` A sequence of one or more words enclosed in angular brackets is called a non-terminal symbol and is used to name a syntactic construct of the language. Each non-terminal symbol is defined in terms of terminal and non-terminal symbols. Some conventions which help in readability have been adopted:

- A prefix of `OPT` designates a non-terminal which is optional when used on the right hand side of a definition.
- The suffix `LIST` denotes a list of objects separated by commas.
- The suffix `SEQ` denotes a list of objects separated by one or more blanks.

`::=` This symbol is used in the definition of non-terminal symbols. The non-terminal symbol appearing on its left is defined to consist of the sequence of terminal and non-terminal symbols appearing on its right.

`|` This symbol is used to separate alternate definitions of a non-terminal symbol.

`" "` A pair of double quotes enclosing one or more characters indicates that the enclosed characters are to appear literally. This allows the use of the meta-symbols as actual symbols in the language.

EXAMPLES: The following describes the syntax of a procedure call that might appear in some simple language:

```
<PROCEDURE CALL>    ::= <PROCEDURE NAME> <OPT PARM LIST>

<PROCEDURE NAME>    ::= <ID>

<ID>                ::= <LETTER>
                      | <ID> <LETTER OR DIGIT>

<LETTER OR DIGIT>    ::= <LETTER>
                       | <DIGIT>

<LETTER>             ::= A | B | C

<DIGIT>              ::= 1 | 2 | 3

<OPT PARM LIST>      ::=
                       | ( <ID LIST> )
```

```
<ID LIST>            ::= <ID>
                       | <ID LIST> , <ID>
```

Note that the first definition for <OPT PARM LIST> is empty indicating that it is optional. The above describes a procedure call to be:

> A procedure name which is an identifier, optionally followed by one or more identifiers separated by commas and enclosed in brackets. An identifier is a letter followed by zero or more letters or digits. The allowable letters and digits are A, B, C and 1, 2, 3 respectively.

Valid procedure calls would be:

```
AB
B123 (ABC)
C(A3, B2, C1)
```

Note that the non-terminal symbol <ID LIST> is used to recursively define itself.

A complete set of definitions in the above notation constitutes the grammar of a language. The grammar for PROTEL is in "APPENDIX B: Language Grammar".

3.0 Basic Structure of the Language

3.1 Language Format

The language source is free-form input between columns 1 and 110 (inclusive) of the input record. Embedded blanks are permitted anywhere except within numbers, identifiers, keywords or special symbols.

3.2 Keywords

The keywords of the language consist of all the multiple letter terminal symbols. Keywords are reserved and may not be used as identifiers. Each keyword must be delimited by a non-alphanumeric character. The complete list of keywords for the language appears in Figure 6. In the examples given in this manual keywords are capitalized for clarity.

3.3 Comments

Comments are prefixed by a percent sign (%) and extend to the end of the input record. They may begin anywhere in the record and may be preceded by source text on the same line.

3.4 Special Symbols

The terminal symbols, other than keywords, constitute the special symbols of the language. A complete list of the special symbols appears in Figure 7.

4.0 Data Elements

4.1 Identifiers

Identifiers in the language are strings of characters used to name data types, constants, variables, procedures, statement labels and sections. Identifiers are represented by the non-terminal <ID> in the language syntax and consist of a letter or special character ($_) followed by 0 to 31 alphanumeric (letters or digits) or special characters.

In the original implementation, case was not significant. However, in PROTEL 2026, the case is significant by default. This may be changed as a compiler option.

EXAMPLES

```
HI_THERE    AMOUNT_IN_$    NUMBER35    $AMOUNT    LastName
```

4.2 Numbers

Decimal numbers are represented by a sequence of decimal digits. Hexadecimal numbers are denoted by the symbol # followed by a sequence of hexadecimal digits (digits 0 to 9, letters A to F). Negative decimal numbers are prefixed by a minus sign. The non-terminals <DECIMAL NUMBER> and <HEX NUMBER> are used to denote decimal and hexadecimal numbers in the language syntax.

EXAMPLES

```
100  1   -17   #A     #7FFF
```

4.3 Simple Constant Expressions

SYNTAX

```
<CONSTANT EXPRSN> ::= <EXPRSN>
```

EXAMPLES

```
5

2 * (table_size - 1)
% where table_size is a constant

{red, blue} INCL hue
% boolean valued constant expression
% if hue is a constant set
```

DESCRIPTION: A constant expression is an expression as described in "Expressions" which can be evaluated at compile time.

4.4 Aggregate Constants (Tuples)

SYNTAX

```
<TUPLE> ::= <OPT REPETITION FACTOR> {<OPT TUPLE BODY LIST>}
          | <OPT REPETITION FACTOR> <STRING>

<OPT REPETITION FACTOR> ::=
                            | <PRIMARY>

<OPT TUPLE BODY LIST> ::=
                          | <TUPLE BODY LIST>

<TUPLE BODY LIST> ::= <EXPRSN>
                      | <TUPLE BODY LIST>,<EXPRSN>
```

EXAMPLES

1. {1, 3, 5}

2.
```
{size, length - 1, 2 << 11}
% the symbol << is a bit shift left symbol; in the above
% example the  binary value for two  is being shifted 11
% positions, which is equivalent to 4096.
```

3.
```
3{2, 4}
% equivalent to {2,4,2,4,2,4}
```

DESCRIPTION

1. A tuple consists of a sequence of expressions surrounded by braces. The expressions may be evaluated at compile time or may require run time evaluation. Note that expressions may themselves be tuples. Thus, the nesting of tuples is allowed.

2. A tuple may be preceded by a repetition factor. The value of <OPT REPETITION FACTOR> must be an integer whose value can be determined at compile time. A zero repetition factor is equivalent to the null tuple, {}. A non-zero repetition factor is equivalent to a single tuple in which the <TUPLE BODY LIST> is repeated the indicated number of times. If no <OPT REPETITION FACTOR> is present, the repetition is assumed to be 1.

4.5 Character Strings

A character string is a sequence of characters enclosed in single quotes (apostrophes). If a user wishes to use a single quote in a string then that quote should be repeated. A repetition factor similar to the one used with tuples may precede a string.

EXAMPLES

```
'ABC'
'DON''T'
2'MAU'     % equivalent to 'MAUMAU'
```

The type of a <STRING> is a tuple of numeric constants whose values are the binary values corresponding to each character in <STRING>.

EXAMPLES: On a machine which uses the ASCII character set:

```
'A'      % is equivalent to the number #41 or 65 (decimal).
'AB'     % is the tuple {#41,#42} or {65,66} (decimal)
```

5.0 Unit of Compilation

SYNTAX

```
<COMPILATION UNIT> ::= <SECTION>

<SECTION> ::= <SECTION HEAD> <OPT STMT SEQ>

<SECTION HEAD> ::= <MODULE TYPE> <SECTION TYPE> <ID>
                   <OPT PACKAGE> <OPT USES_IMPL LINK> ;

<MODULE TYPE> ::=
                 | FAST
                 | PERPROCESS
                 | DEFINITIONS

<SECTION TYPE> ::= INTERFACE
                  | SECTION

<OPT PACKAGE> ::=
            | IN <ID>

<OPT USES_IMPL LINK> ::=
                       | USES <ID LIST>
                       | OF <ID LIST>

<OPT STMT SEQ> ::= <OPT DCL STMT SEQ> <OPT EXEC STMT SEQ>
```

EXAMPLES

1.

```
INTERFACE system USES nucleus;
 .
 .
 .
% end_of_file
```

2.

```
SECTION translation OF system;
 .
 .
 .
% end_of_file
```

3.

```
SECTION statistics USES system, ama;
 .
 .
 .
% end_of_file
```

4.

```
DEFINITIONS INTERFACE defs;
  .
  .
  .
% end_of_file
```

5.

```
SECTION kernel;
  .
  .
  .
% end_of_file
```

DESCRIPTION

1. The section is the basic unit of compilation. It is declared using the keyword SECTION or INTERFACE. As a result of compiling a section, an object file is produced containing an identifier symbol table, object code for all executable statements and initialized data areas.

2. The module is the basic logical building block of software: it consists of one or more sections. A module whose top-level sections are declared using the keyword INTERFACE may be used by other modules. Otherwise, the module may not be used since it provides no interface.

   (See description of the USES LINK below.

3. When a section y USES a section z, or is OF section z, we say that y imbeds z. We say that z outbeds all of its global identifiers. Thus the imbedding of z by y causes y to imbed those identifiers outbedded by z.[1]

4. Interface sections can only imbed other interface sections. For example, the module structure described by the following declarations is not allowed.

```
INTERFACE top;

SECTION second OF top;

INTERFACE third OF second;
```

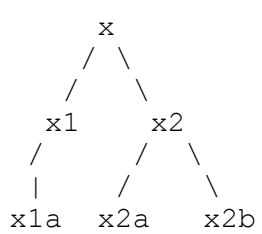
[1] Imbedding and outbedding are discussing more fully in the chapter on PROTEL Program Organization of the PROTEL Introductory Manual.

5. Only one linear chain of interfaces is permitted in each module. For example, the module structure described by these declarations is not allowed since x1 and x2 are on different branches from x.

```
INTERFACE x;

INTERFACE x1 OF x;
SECTION x1a OF x1;

INTERFACE x2 OF x;
SECTION x2a OF x2;
SECTION x2b OF x2;
```



6. The OF <ID LIST> phrase of the SECTION or INTERFACE statement specifies previously compiled sections which the current section is to logically imbed. This imbedding allows the current section to gain access to items defined in the previously compiled sections. The imbedding of sections in a module is indicated by the keyword OF. Imbedding links within a module are transitive, e.g., in the examples above, section "translation" has access to all variables, constants, types, and procedures known to section “system".

7. The USES <ID LIST> phrase can only appear on the SECTION or INTERFACE statement of the top-level section of a module. It specifies previously compiled interface sections of other modules to be logically imbedded into the current module. This use of one module by another is denoted by the keyword USES.

   NOTE: USES links are non-transitive, except for some type names: Type names which are referenced in an interface are automatically outbedded with the interface.

8. If more than one <ID> follows USES, the current module is considered to imbed all the interfaces denoted by the list of <ID>s.

9. Circular imbedding is not allowed. (i.e., the USES graph must be acyclic.)

10. The <OP PACKAGE> ("IN packagename") may only be used in the top most section of a module (i.e., a section whose section head contains no "OF" list). When present, it specifies that the imbedding restrictions specified for that package (in an external configuration file) are to be enforced for the module.[2]

---

[2] This facility is not currently used by any DMS PROTEL programs. Package enforcement is provided by the PLS library system.

6.0 Module Types

There are four types of modules: FAST, SWAPPABLE, PERPROCESS, and DEFINITIONS.

Modules declared with the FAST keyword or with no keyword (called swappable) contain only protected and shared data (see "Data Declarations"). In the DMS implementation, a fast module is allocated dedicated base registers which do not need to be "swapped" during procedure calls to its procedures. Note that, in the IBM/370 implementation, all modules are treated as FAST.

The keyword PERPROCESS declares a module which contains protected, shared, and private data.

The keyword DEFINITIONS declares a module which contains only type and constant declarations. No storage is allocated and no code is generated for this type of module.

Figure 1 summarizes which types of modules may use which other types.

USES	FAST	SWAPPABLE	PERPROCESS	DEFINITIONS
FAST	T	T	F	T
SWAPPABLE	T	T	F	T
PERPROCESS	T	T	T	T
DEFINITIONS	T	T	T	T

Figure 1. Valid relationships: which types of modules may use which other types. (Module type on "row-left" USES module type on "column-top".)

7.0 Declarations

7.1 Type Declarations

A type declaration associates an identifier with a description of the properties which instances of that type possess. These properties include the set of values the instance may assume, the amount of storage it requires and whether its value may be updated. (The properties of procedure types are slightly different and are dealt with in a separate section.)

SYNTAX

```
<TYPE DCL STMT> ::= TYPE <TYPE DCL LIST>

<TYPE DCL LIST> ::= <TYPE DCL>
                  | <TYPE DCL LIST>, <TYPE DCL>

<TYPE DCL> ::= <ID LIST> <TYPE>

<TYPE> ::= <OPT PACKING> <OPT VAL> <UNPACKED TYPE>

<OPT PACKING> ::=
                | PACK ( <CONSTANT EXPRSN> )

<UNPACKED TYPE> ::= <SIMPLE TYPE>
                  | <AGGREGATE TYPE>
                  | <PROC TYPE>

<OPT VAL> ::=
            | VAL
```

DESCRIPTION

1. The TYPE declaration allows the definition of new named data types from the existing primitive types of the language or from other types defined by the user. The primitive data types consist of the simple types, denoting a single value, and aggregate types, denoting collections of values and the procedure types. These types are described later in this chapter.

2. The identifiers appearing in <ID LIST> are user-defined types and may be used in any context where a type is required. A type may be defined in terms of other types as long as circular definitions do not occur, e.g.,

```
TYPE  a  b;  TYPE  b  a;   % NOT ALLOWED
```

3. The VAL attribute, if present, indicates that an instance of the type may not appear in any context where its value may be updated.

4. The <OPT PACKING> phrase permits control over the packing of data into computer words. The use of packing is described in section "PACK and VAL Attributes".

5. The definition of a defined type need not appear before the type is referenced in another declaration statement. However, the definition should occur before an instance of the type is referenced. When the definition of a type occurs after the type has been used, we refer to this as a forward type. The use of forward types is strongly discouraged.[3]

EXAMPLE

```
DCL i t;                % creates an instance i of type t.
                        % i is global.

DCL  p  PROC() IS       % declaration for procedure p.
   i + 1 -> i;          % reference to global i.

TYPE t {0 TO 255};      % definition of type t. This
                        % definition should have
                        % appeared before the
                        % declaration of i.
```

6. Types are used, amongst other reasons, to insure that operations are performed on the right data items and that compatibility exists between the operands of an operation. The rules for type checking and compatibility are in section "Type Compatibility".

[3] This is not the same as an IS FORWARD procedure declaration.

7.1.1 Simple Types

SYNTAX

```
<SIMPLE TYPE> ::= <RANGE TYPE>
                | <BOOL TYPE>
                | <PTR TYPE>
                | <DEFINED TYPE>

<RANGE TYPE> ::= <NUMERIC RANGE>
               | <SYMBOLIC RANGE>
```

DESCRIPTION

1. The simple types denote a single value. They consist of the range types, which are subdivided into numeric and symbolic ranges, the BOOL type, the PTR (pointer) type and the defined type.

7.1.1.1 Numeric Ranges

SYNTAX

```
<NUMERIC RANGE> ::= {<CONSTANT SUBRANGE>}

<CONSTANT SUBRANGE> ::= <CONSTANT EXPRSN> TO <CONSTANT EXPRSN>
```

EXAMPLES

```
TYPE integer {-32768 TO 32767};
     % Defines type integer to be a number in the range
     % -32768 to 32767, i.e., the maximum range of a 16 bit
     % signed integer in two's complement arithmetic.

TYPE pb_in_page, page_of_pbs {0 TO 255}, % both have
                                         % the same range.
     interface_unit {0 TO 1023},
     counter {1 TO 100};

TYPE client_table_index {0 TO client_table_size - 1};
     % Shows use of a constant expression in
     % <CONSTANT SUBRANGE>.

TYPE unsigned_integer {0 TO 65535};
     % Defines type unsigned_integer to be the maximum
     % value that may be represented in a sixteen bit
     % word.
```

DESCRIPTION

1. Numeric ranges are used to define types whose instances may assume a numeric value. The range is used to determine the amount of storage required for an instance of that type.

   NOTE: Currently the DMS implementation performs range checking on simple assignments to insure that an instance of a numeric type is assigned only values within its range. The current implementation checks that constant assignments are in range, and that variable-to-variable assignments are between variables with overlapping ranges. This range checking is performed at compile-time only; there is no run-time range checking in the DMS implementation.

   The IBM/370 implementation can perform range checking of simple assignments both at compile- and run-time.

2. The type integer is not a built-in type in the language. The example above shows the definition of integer for a sixteen bit word in signed two's complement arithmetic.

7.1.1.2 Symbolic Ranges

SYNTAX

```
<SYMBOLIC RANGE> ::= {<ID LIST>}

<ID LIST> ::= <ID>
            | <ID LIST>, <ID>
```

EXAMPLES

```
TYPE weekday {mon, tues, wed, thurs, fri};

TYPE stat1, stat2 {busy, blocked, idle, ready},
     class {two_party,mlhg,manual,unassigned,denied};
```

DESCRIPTION

1. A symbolic range specifies by enumeration the allowable constant values that may be assumed by an instance of the type. For example, a variable day of type weekday may only be assigned one of the values mon, tues, wed, thurs, or fri.

2. Each identifier in a symbolic range (each <ID> in <ID LIST>) must be different from all other identifiers known in the current scope. For scope rules see “Scope of Identifiers”.

3. The elements of a symbolic range are considered to be ordered in the order of enumeration for the purpose of executing a loop over all the elements of the range (see "Iterative Statement").

4. Internally, symbolic range elements are mapped onto the numbers 0 to n-1 where n is the cardinality of the range.

7.1.1.3 Boolean

SYNTAX

```
<BOOL TYPE> ::= BOOL
```

EXAMPLES

```
TYPE end_of_file BOOL;

TYPE active_cpu, sane, blocked BOOL;
```

DESCRIPTION

1. The type BOOL is used to represent logical quantities. An instance of type BOOL may only assume a value of TRUE or FALSE.

2. The identifiers TRUE and FALSE are predefined constants in the language and may only be used as constants of type BOOL.

7.1.1.4 Pointers

SYNTAX

```
<PTR TYPE> ::= PTR TO <TYPE>
```

EXAMPLES

```
TYPE q_head PTR TO q_element;

TYPE p, g PTR TO feature_block;
     %  An item of type feature_block may be pointed
     %  to by items of type p and g.
```

DESCRIPTION

1. An item of type pointer (PTR) contains an absolute address and is used to indirectly reference data.

2. Operators are defined in the language to retrieve the address of a data item (PTR operator) or to allow access to an item pointed at (@ operator).

3. The pointer constant NIL denotes a null value. It may be assigned to any pointer.

4. A pointer may only point to an instance of the type given in the pointer declaration. For example, a pointer to type X may not be assigned the address of an instance of type Y.

5. Pointers may only reference items which are aligned on an addressable boundary.

7.1.1.5 Defined Types

SYNTAX

```
<DEFINED TYPE> ::= <ID>
```

EXAMPLES

```
TYPE int integer;
     % defines int to be the numeric range
     % {-32768 to 32767}

TYPE mon_to_fri weekday;
     % defines mon_to_fri to be the symbolic
     % range weekday.
```

DESCRIPTION

1. Such definitions define a user-defined type to be another user defined type. The <ID> in the definition must be a type name.

2. Note that there are no predefined numeric or symbolic types in the language. In particular, integer, char and string are not predefined.

7.1.2 Aggregate Types

SYNTAX

```
<AGGREGATE TYPE> ::= <SET TYPE>
                   | <STRUCT TYPE>
                   | <TABLE TYPE>
                   | <DESC TYPE>
                   | <AREA TYPE>
```

DESCRIPTION

1. If two objects are of the same type, we say they are homogeneous. If two objects are not of the same type, we say they are heterogeneous or non-homogeneous.

2. Aggregate types are used to define collections of elements organized in some fashion. An aggregate type is characterized by the type of its elements and its organization method. Collections of homogeneous elements are grouped into tables, descs or sets. Collections of non- homogeneous elements are grouped into structs or areas.

7.1.2.1 Sets

SYNTAX

```
<SET TYPE> ::= SET <SIMPLE TYPE>
```

EXAMPLES

```
TYPE bit_word SET {0 TO 15};
      % Allows bits in a word to be set or cleared
      % individually or in groups. An individual bit
      % can be referenced by its bit position 0 to 15.

TYPE hue SET {red, green, blue};
```

DESCRIPTION

1. The type SET is used to denote a data item which may assume values consisting of any subset of values from a symbolic or numeric range.

   For example, an instance of type hue above may assume any of the values: {}, {red}, {green}, {blue}, {red, green}, {red, blue}, {green, blue}, or {red, green, blue}.

2. Within an instance of a SET, one bit is associated with each element of the symbolic or numeric range.

3. The maximum cardinality of a set type is implementation-dependent, and may not exceed the number of bits in a word. Consequently, symbolic ranges used in sets may not have a cardinality greater than 16 in the DMS implementation, and 32 in the IBM/370 implementation. Numeric ranges used as set base types may not have an upper bound greater than 15 (DMS) or 32 (IBM/370), since the lower bound is taken as zero.

7.1.2.2 Structures

SYNTAX

```
<STRUCT TYPE> ::= STRUCT <DCL LIST> ENDSTRUCT

<DCL LIST> ::= <ID LIST> <TYPE>
             | <DCL LIST> , <ID LIST> <TYPE>
             | <OVLY GROUP>
```

EXAMPLES

1.
```
TYPE task_block
        STRUCT
           priority    {0 TO 5},
           status      {ready, blocked, executing},
           next_task   PTR TO task_block,
           completed   BOOL,
           code        code_block % defined type
        ENDSTRUCT;
```

2.
```
TYPE nested_struct
       STRUCT
          i  integer,
          ss     STRUCT    % substructure
                   b  BOOL,
                   r  {x,y,z}
                 ENDSTRUCT,
          p      PTR TO int
       ENDSTRUCT;
```

DESCRIPTION

1. A structure type (STRUCT) is used to build a collection of heterogeneous items. The STRUCT type definition associates a name with the structure type and a name and type with each element appearing in the structure.

   Items within a structure may be of any type whether simple or aggregate. Thus, it is possible to define nested structures. Note that two elements of the same structure may not have identical names unless they appear in different substructures.

2. An item within a structure is denoted by the name of an instance of the structure qualified by the name of the desired item. If the item is within a nested structure, all substructure names must appear in the denotation.

EXAMPLES

```
instance_task_block.code
instance_nested_struct.ss.r
instance_nested_struct.i
```

3. <OVLY GROUP> is discussed in "Structure Overlays".

7.1.2.3 Tables

SYNTAX

```
<TABLE TYPE> ::= TABLE <BOUNDS> OF <BASE TYPE>

<BOUNDS> ::= [ <CONSTANT SUBRANGE> ]
           | [ <DEFINED TYPE> ]

<CONSTANT SUBRANGE> ::= <CONSTANT EXPRSN> TO <CONSTANT EXPRSN>

<DEFINED TYPE> ::= <ID>

<BASE TYPE> ::= <TYPE>
```

EXAMPLES

1.
```
TYPE t TABLE[0 TO 5] OF {1 TO 128};
```

2.
```
TYPE day_count TABLE[day] OF counter;
```

3.
```
TYPE matrix TABLE[1 TO 20] OF TABLE [0 TO 7] OF integer;
```

DESCRIPTION

1. A table type is used to build a named linear collection of homogeneous items. The type definition consists of two major parts: A <BOUNDS> declaration and a <BASE TYPE> declaration. The <BOUNDS> indicates the size of the table and the type of the indices; the <BASE TYPE> defines the type of the elements in the table.

2. <BOUNDS> may be either a constant subrange or a defined type. If <BOUNDS> is a defined type then it must be either a symbolic or numeric range. The number of elements in the table is the number of elements in the range of <BOUNDS>.

3. The <BASE TYPE> of a table may be any type, whether simple or aggregate. Thus, it is possible to define a two dimensional table as a table of table. This situation is illustrated in the third example above.

4. A table element is accessed by specifying the name of the table and an index. An index is a bracketed expression which indicates the position of the desired element within the table. Note that the index has to be of the same type as the bounds. Elements of a table of more than one dimension are accessed by specifying the name of the table followed by as many indices as there are dimensions.

EXAMPLES

```
instance_t[4]
instance_day_count[wed]
instance_t[4 * i - 1]
instance_matrix[10][i]
```

A check is made to insure that the value of an index does not lie outside the range of the corresponding bounds.

5. All PROTEL compilers consider the symbols (| and [as equivalent, and the symbols |) and] as equivalent.

   Within character strings, however, they are not the same. In particular, the case selector labels {'['}: and {'(|'}: are not equivalent.

7.1.2.4 Descriptors

SYNTAX

```
<DESC TYPE> ::= DESC <OPT BOUNDS> OF <BASE TYPE>

<OPT BOUNDS> ::=
                | <BOUNDS>

<BOUNDS> ::= [ <CONSTANT SUBRANGE> ]
           | [ <DEFINED TYPE> ]

<CONSTANT SUBRANGE> ::= <CONSTANT EXPRSN> TO <CONSTANT EXPRSN>

<DEFINED TYPE> ::= <ID>

<BASE TYPE> ::= <TYPE>
```

EXAMPLES

1.
```
TYPE d DESC OF data_block;
```

2.
```
TYPE e DESC[0 TO 9] OF integer;
```

3.
```
TYPE op_name TABLE[opcode] OF DESC OF char;
```

4.
```
TYPE string DESC OF char;
```

DESCRIPTION

1. Descriptors are used to provide indirectly referenced tables in which the table bounds or storage location may be modified at run time. They also provide a mechanism for implementing procedures which can accept tables of any size as a parameter.

2. A <DESC TYPE> declaration is similar in most ways to a <TABLE TYPE> declaration. If <BOUNDS> is omitted from the declaration, the lower bound is assumed to be zero. The upper bound will vary at run time depending on the information denoted by the descriptor.

3. Conceptually, a descriptor consists of two parts: The descriptor part and the table part.

   - The descriptor part contains a pointer to the base of the table, and the number of elements in the table. It may also, depending upon the PROTEL implementation, contain

a stride field. The "stride" of a descriptor is the width of (i.e., amount of storage required by) each element in the table part.[4]

- The table part contains the elements of the table. At run time, the information in the descriptor part is modified to reflect changes in the number of elements in a table or to point to different tables.

4. The operator UPB allows access to the value of the upper bound in a descriptor or table. The value returned is of the same type as the bounds. The operator TDSIZE allows access to the number of elements in a table or descriptor. The value returned is a numeric type. These will be discussed in more detail in "Structure of an Expression".

5. Descriptors may be assigned the value NIL to indicate that they describe no data. Attempting to index a NIL descriptor is an error.

6. The declaration of a descriptor variable does not allocate storage for the table part. It allocates storage for the descriptor part, which is, in effect, a pointer to a table. Until assigned a value, the descriptor does not point to any table. A local descriptor is initialised to NIL.[5]

---

[4] In the current MC68020 implementation, a descriptor does not contain a stride field; in the NT40 implementation a stride field is present.

[5] Because of this, a local descriptor declaration is executable.

7.1.2.5 Areas

SYNTAX

```
<AREA TYPE> ::= <AREA HEADER> <AREA BODY> ENDAREA

<AREA HEADER> ::= <AREA OPTIONS> AREA ( <EXPRSN> )
                | <AREA OPTIONS> AREA REFINES <DEFINED TYPE>

<AREA OPTIONS> ::=
                 | UNRESTRICTED

<AREA BODY> ::=
              | <DCL LIST>
```

EXAMPLES

1.
```
TYPE father UNRESTRICTED AREA(3 * 16)
          i int
     ENDAREA;
```

2.
```
TYPE son1 AREA REFINES father
          t TABLE[0 TO 15] OF BOOL
     ENDAREA;
```

3.
```
TYPE son2 AREA REFINES father
          p PTR TO int
     ENDAREA;
```

DESCRIPTION

1. The area type (AREA) is used to build a collection of heterogeneous items, similar to those built by a structure type. The major difference between structures and areas is that areas provide a facility to postpone declaration of items in the structure to a later time.

2. An AREA is a father area or a refinement of a father area. The declaration of a father area indicates the amount of storage required for the area and may specify some of the items which make up the area. A refinement of an area indicates the names and types of other items in the area. Several refinements may be associated with the same father area. In such cases, all refinements will be overlayed in storage, after the elements of the father area. A refinement of an area may not specify a size for the area.

3. A refinement of an area may, itself, be a father area. For example

```
TYPE gson AREA REFINES son1
        s weekday
ENDAREA;
```

The storage layout consists of i then t or p then s.[6]

4. In the above example father is a father area and is allocated 3 sixteen bit words. The first word will be occupied by i. Son1 and son2 are refinements of father. The elements of these 'brother' areas are overlayed in the storage which follows (i.e., starting at the 2nd word of the area).

5. The UNRESTRICTED attribute signifies that an area may be refined outside the module in which it is declared. If this attribute is not present, all refinements to the area must be in the same module as the area declaration.

6. An element in an area is denoted in the same manner as an element of a structure.

EXAMPLES

```
instance_son1.t[j]
instance_father.i
```

[6] The fields t and p cannot both be simultaneously visible. However they may both be used in a given instance of father, since the instance may be subject to different refinements at different times.

7.1.3 Procedure Types

7.1.3.1 User Defined Procedures

SYNTAX

```
<PROC TYPE> ::= <PROC> ( <OPT PARMS> ) <OPT RETURNS>

<PROC> ::=  <OPT PCLASS> PROC

<OPT PCLASS> ::=
               | QUICK
               | NONTRANSPARENT

<OPT RETURNS> ::=
                | RETURNS <TYPE>

<OPT PARMS> ::=
              | <PARM LIST>

<PARM LIST> ::= <PARM>
              | <PARM LIST>, <PARM>

<PARM> ::= <OPT PASSING MODE> <ID LIST> <TYPE>

<OPT PASSING MODE> ::=
                     | REF
                     | UPDATES
```

EXAMPLES

1. `TYPE p PROC(i, j integer) RETURNS integer;`

2. ```
   TYPE get_db PROC(UPDATES datablk data_block)
                RETURNS BOOL;
   ```

3. ```
   TYPE sort PROC(REF in_tab tab,
                  UPDATES out_tab tab);
   ```

DESCRIPTION

1. The procedure type declaration is used to define the attributes of a procedure. These include a description of the formal parameters (<OPT PARMS>) and, if any, the return value. The actual body of the procedure is defined using a variable declaration with initialization

or a constant definition (see "Data Declarations".)

2. The declaration of the parameters specifies, for each parameter, its name, its type and its passing mode. Parameters may be of any type.

3. The passing mode may be "by reference" (REF), "by updates" (UPDATES), or "by value" (default).

   If a parameter is passed by value, a local copy of the value is made on procedure entry. Throughout the procedure, references to the parameter are treated as references to the local copy. If the parameter is updated inside of the procedure, the value outside the procedure will not be affected since only the local copy will be changed.

   If a parameter is passed by reference, the address of the actual parameter is passed to the procedure. The parameter, however, is considered to be read-only and may not appear in any context where it can be updated. Reference parameters are intended to avoid making local copies of large aggregates which are passed as parameters.

   If a parameter is passed by updates, it is handled in the same manner as a reference parameter. The difference is that updates parameters may be updated.

4. A procedure defined with a RETURNS clause is a function. The returns clause specifies the type of the item to be returned.

5. A procedure should NEVER return, via an UPDATES parameter or function return, a pointer or descriptor referring to a local value. A local value is either a formal parameter passed by value or an item declared within the body of the procedure. This restriction results from the fact that local values disappear when procedure execution terminates.

6. Procedure variables are fully supported. A procedure variable may be assigned the value NIL to indicate that it describes no procedure.

7.1.3.2 Procedure Classes

Procedures in PROTEL/NT40 are of one of three procedure classes: standard, QUICK and NONTRANSPARENT. The procedure class denotes which type of procedure calling mechanism is required.[7]

EXAMPLES

```
TYPE allocator NONTRANSPARENT PROC();

DCL calc_x QUICK PROC(a, b dint) RETURNS dint IS FORWARD;

DCL swerror PROC() IS FORWARD;  % standard attributes
```

A standard procedure, the default, requires base registers 1, 2, and 3 to be set up before invocation and restored after the procedure returns.

QUICK denotes a procedure that does not reference any data in its head segments. Hence callers do not need to set up registers 1, 2, or 3 before calling, nor do they need to restore the registers after the call. It is an error if a procedure declared as QUICK refers to its head segments. Since procedures in FAST modules don't require base register setup, this attribute is useless for them.

NONTRANSPARENT denotes a procedure that does not need registers 1, 2, or 3 (like QUICK), but that may return with those registers altered. If a procedure in a swappable or perprocess module is declared as NONTRANSPARENT but does in fact refer to its protected or shared head segments, the compiler outputs a hint, generate a LDREG instruction after the BPROC, and compile the procedure as if it were neither QUICK nor NONTRANSPARENT. A count of the number of procedures in which this was done is included in the output of the CSTATS (console statistics) compiler option. It is an error if a perprocess NONTRANSPARENT procedure refers to its private head segment.

The purpose of procedures attributes is to minimize the need to swap registers 1, 2, and 3 in conjunction with calling procedures. Calling into a QUICK procedure means that the registers don't have to be set up before a call, and calling from an NONTRANSPARENT means the registers don't have to be restored after a call.

The attributes QUICK and NONTRANSPARENT apply regardless of whether the procedure is called directly or via procedure variable. Changing the class of a procedure through the use of CAST is unsafe, and will cause unpredictable results.

---

[7] Procedure class has no meaning in PROTEL/68K, due to the nature of the hardware. All procedures are considered "standard", and the procedure class keywords are ignored. PROTEL/370 flags the use of these keywords as errors.

7.1.4 Object-Oriented Types

The original PROTEL-2 OO extensions are still being researched for PROTEL 2026.

Classes extend types with inheritance and methods.

7.1.4.1. Class Declaration

EXAMPLE

```
TYPE classname CLASS REFINES basetype
   class_data ;
   OPERATIONS
      method_decls ;
ENDCLASS;
```

- basetype: $object (root) or another class (single inheritance).
- class_data: Fields with attributes {PROTECTED, SHARED, READABLE, etc.}: field type;.
- Class variables (static fields): Use {class} attribute in data flags. Shared across instances.
- Access via instance.class_var or SELF.class_var.

7.1.4.2 Methods

In OPERATIONS: {attrs}: method_name METHOD (params) RETURNS type;.

- Attributes: {CLASS} (static methods), {OVERRIDING}, {ABSTRACT}, {CREATE} (constructors), etc.
- Class methods: Invokable on class name (e.g., ClassName.static_method()). SELF refers to class.
- Instance methods: Use SELF for this. No explicit receiver param (implicit handling).
- Overrides: Marked {OVERRIDING} in derived classes.

EXAMPLE

```
TYPE BaseClass CLASS REFINES $object
   {READABLE}:
      id $longint;
   {PROTECTED, class}:
      count $int;  % Static field
OPERATIONS
   {CLASS}:
      static_method () RETURNS $int;
   init_me METHOD (UPDATES);
   {FIXED, CREATE}:
      new METHOD (cd $classdesc, st storetype) RETURNS PTR TO ANY
BaseClass;
   {OVERRIDING}:
      virtual_method METHOD() RETURNS $int;
ENDCLASS;
```

- Method Implementation: (In SECTION OF interface)

```
IN classname DCL method_name METHOD IS
BLOCK
   ... body ...  % Use SELF.id, etc.
ENDBLOCK;
```

- Instantiation: ClassName.new(params) -> ptr;. Calls on pointers: obj.method().

7.2 Data Declarations

Data consists of either variables or constants. A variable has a value which may change at run time whereas a constant has a value, known at compile time, which is fixed throughout a program. Constants may either be self-defining or declared. Variables and declared constants, unlike types, require declarations before they may be used.

```
<DATA DCL STMT> ::= <VARIABLE DCL STMT>
                  | <CONST DCL STMT>
```

7.2.1 Variables

SYNTAX

```
<VARIABLE DCL STMT> ::= PROTECTED <VARIABLE DCL LIST>
                      | SHARED <VARIABLE DCL LIST>
                      | PRIVATE <VARIABLE DCL LIST>
                      | DCL <VARIABLE DCL LIST>

<VARIABLE DCL LIST> ::= <VARIABLE DCL>
                      | <VARIABLE DCL LIST>, <VARIABLE DCL>

<VARIABLE DCL> ::= <ID LIST> <TYPE> <OPT DATA INIT>

<OPT DATA INIT> ::=
                  | INIT <UNLABELLED STMT>

<UNLABELLED STMT> ::= <ITERATIVE STMT>
                    | <IF STMT>
                    | <CASE STMT>
                    | <EXIT STMT>
                    | <RETURN STMT>
                    | <BLOCK>
                    | <EXPRSN>
```

EXAMPLES

1.
```
DCL a, b char;
```

2.
```
DCL i, j {0 TO 127};
```

3.
```
DCL counter {0 TO 999} INIT 0;
```

4.
```
DCL tab_size {0 TO max_size} INIT size + 1;
```

5.

```
DCL max_q_size TABLE[q_type] OF int
               INIT {5{10}, 6, 3};
```

6.

```
DCL sp_features
      SET {digitone,call_xfer,add_on,abr_dial}
      INIT {};
      % Note null initialization
```

7.

```
DCL label_stack TABLE[stack_index] OF
    STRUCT
       value    int,
       label    internal_label
    ENDSTRUCT;
```

8.

```
DCL s
   STRUCT
      a    {-10 TO 10},
      b    {x,y,z},
      c  STRUCT
            d    TABLE[0 TO 5] OF {0 TO 15},
            e    BOOL,
            f    TABLE[0 TO 3] OF char
         ENDSTRUCT,
      h    PTR TO q
   ENDSTRUCT
      INIT {-10,x,{6{0}, FALSE, 'pqrs'},NIL};
```

9.

```
DCL p PROC(i integer, b DESC OF weekday)
      IS RETURN;
```

DESCRIPTION

1. The data declaration statement is used to create an instance of a type and to associate a name with that instance. Storage will be allocated for each instance created.

2. There are two types of declarations: local and global. A local declaration is one that occurs within a block and whose effect is limited to that block. A global declaration occurs in a section outside of any block, and its effect is limited to that section, from its point of declaration down, and to any section which imbeds it.

   The DCL keyword may be used to declare both local and global variables. The PROTECTED, SHARED and PRIVATE keywords may only be used to declare global variables. They specify the type of storage segment the declared variables are allocated in.[8]

   PROTECTED declares constants and read-only variables.

---

[8] They are meaningless in PROTEL/370; all variables are considered SHARED.

SHARED declares variables to be shared among all instances of a program.

PRIVATE declares per-process (non-shared) variables. Each process using such variables has its own instance of the variables.

The DCL keyword, when used in a PERPROCESS module, is equivalent to the PRIVATE keyword. In all other types of modules, it is equivalent to the SHARED keyword.

3. The <TYPE> used in a variable declaration can be a defined type created with a TYPE declaration or a full type definition. A variable not declared in terms of a defined type has an anonymous type. Caution should be exercised when using a full type definition in a variable declaration as it may then be impossible to create a variable whose type is compatible with the first. The type compatibility rules of the language are detailed in "Type Compatibility".

4. The <OPT INIT> clause allows an initial value to be assumed by a variable at the time of its allocation to storage. For variables at the global level (i.e., not inside any procedure block) this initialization takes place only once when memory is loaded by the loader.[9] For local variables (i.e., those declared within some block), the initialization takes place on block entry.

5. The <UNLABELLED STMT> following INIT consists of an executable statement or an expression (<EXPRSN>). These items are discussed in detail in later sections of the manual. The executable statements may be specified only inside the body of a procedure. In this case the statements are compiled into machine code.

   When initializing an aggregate, <EXPRSN> may be in the form of a tuple. Each element of the aggregate is initialized by one element of the tuple. If an element of an aggregate is itself an aggregate, the corresponding tuple element must be a tuple. Note in example 7 above that the inner structure, c, is initialized with the tuple:

```
{6{0}, FALSE, 'pqrs'}
```

   Within the above tuple 6{0} is equivalent to {0,0,0,0,0,0} and 'pqrs' is equivalent to {‘p','q','r','s'}.

---

[9] In PROTEL/DMS global initialization is restricted to PROTECTED variables which are large enough that, were they constants, would be allocated storage in the protected head segement. In other words, they must have a width of at least 48 bits. Global procedure variables may not be initialized by an INIT clause.

7.2.2 Constants

SYNTAX

```
<CONST DCL STMT> ::= PROTECTED <CONST DCL LIST>
                   | DCL <CONST DCL LIST>

<CONST DCL LIST> ::= <CONST DCL>
                   | <CONST DCL LIST>, <CONST DCL>

<CONST DCL> ::= <ID LIST> <TYPE> IS <CONST DEF>

<CONST DEF> ::= <UNLABELLED STMT>
               | FORWARD
               | ENTRY
               | INTRINSIC

<UNLABELLED STMT> ::= <ITERATIVE STMT>
                    | <IF STMT>
                    | <CASE STMT>
                    | <SELECT STMT>
                    | <EXIT STMT>
                    | <RETURN STMT>
                    | <BLOCK>
                    | <EXPRSN>
```

EXAMPLES

1.
```
DCL seconds_per_hour integer IS 3600;
   % integer is assumed to be a defined
   % type of suitable range.
```

2.
```
DCL primes TABLE[1 TO 10] OF integer IS
             {2,3,5,7,11,13,17,19,23,29};
```

3.
```
DCL header DESC OF char IS
                'Experimental Results follow';
```

4.
```
DCL max_len {128 TO 1024} IS 256;
```

5.
```
DCL op_names TABLE[opnames] OF DESC OF char
             IS {'pushc', 'popc', 'pushci', 'add'};
```

6.
```
DCL forget_onhook PROC() IS FORWARD;
```

7.

```
PROTECTED max PROC(a, b int) RETURNS int IS
   IF a > b       % single statement procedure
      THEN RETURN a;
      ELSE RETURN b;
   ENDIF;
```

8.

```
DCL preempt PROC() IS
     BLOCK
        cptime -> pvp@.ptime;
        henq(cproc,rdq[cprior], plink);
        schedule();
     ENDBLOCK preempt;
```

9.

```
DCL channel_mask channel_wd IS {0,1,
```

DESCRIPTION

1. Constant declarations are used to associate a name with simple, aggregate or procedure constants. The value of a declared constant, unlike the value of an initialized variable, is fixed at compile time and may not be modified during program execution.

2. Only the keywords DCL and PROTECTED can be used to declare constants.

3. Simple and Aggregate Constants

   a. The <UNLABELLED STMT> following IS consists of a constant expression (<EXPRSN>).
   b. The value of <EXPRSN> must be a constant which can be evaluated at compile-time.
   c. Tuples may be used in the definition of constants.

4. Procedure Constants

a. An executable statement is used to define the body of a procedure constant to be a statement (as in examples 6 and 7 above). The declaration of procedure constants is the primary mechanism for defining procedures in the language. The executable statements in the procedure constant declaration are compiled into a block of executable code which corresponds to the procedure constant name.

b. The clauses, IS FORWARD, IS ENTRY, and IS INTRINSIC are used only when defining procedure constants.

   1) The IS FORWARD clause indicates that the body of the procedure declared in the IS FORWARD statement will be found subsequently (i.e., forward of the current statement) in the current module. Note that the forward declaration of a procedure and its body must be in the same module.

   2) The IS ENTRY clause defines a procedure to be the entry point to the current module. A process created from this procedure begins execution by invoking the ENTRY procedure. There can only be one ENTRY procedure per module and its body may be declared in any section of the module.

The declaration of a procedure as ENTRY implies that it IS FORWARD, thus one may not declare a procedure as both.[10]

3) INTRINSIC procedures are discussed in “Intrinsic Procedures".

[10] PROTEL/370 does not support the IS ENTRY clause. It always assumes an entry procedure named "protel".

7.2.3 Pack and Attributes

SYNTAX

```
<TYPE> ::= <OPT PACKING> <OPT VAL> <UNPACKED TYPE>

<OPT PACKING> ::=
                | PACK ( <CONSTANT EXPRSN> )

<OPT VAL> ::=
            | VAL
```

EXAMPLES

1. ```
   DCL s
           STRUCT
             a    BOOL,
             b    PACK(4)  SET{x,y,z},
             c    PACK(32)
                    STRUCT
                      d    int,
                      e    PACK(16) {0 TO 4095}
                    ENDSTRUCT
           ENDSTRUCT;
   ```

2. ```
   DCL t1 TABLE[0 TO 99] OF PACK(4) BOOL;
   ```

3. ```
   DCL t2 TABLE[class] OF PACK(48) class_structure;
   ```

4. ```
   DCL print_string PROC(string DESC OF VAL char) IS FORWARD;
   ```

DESCRIPTION

1. The width in bits of a data element is determined from the element type. In the definition of a type, the PACK attribute may be used to specify a greater width.

EXAMPLES

```
TYPE small_int PACK(4) {0 TO 6};
     % PACK(4) is associated
     % with type small_int

TYPE index_range {0 TO 11};

DCL t1 TABLE[index_range] OF small_int;
     % PACK(4) results in an array of 4-bit elements.
     % t1 is has a width of 48 bits.
```

2. The PACK attribute associated with a type may be overridden by a subsequent PACK appearing in another declaration provided that the width of the type is not decreased by the subsequent PACK.[11] For example in the declaration:

```
DCL t2 TABLE[index_range] OF PACK(8) small_int; % legal
```

small_int is defined in the previous example with PACK(4). The PACK(8) attribute overrides the PACK(4) attribute associated with small_int.

```
DCL t3 TABLE[index_range] OF PACK(3) small_int;
       % Not allowed, since the width of
       % small_int was originally 4.
```

3. If PACK is omitted, the width of a type is determined by the packing rules, which are implementation-dependent. The number of bits allocated to an instance of <UNPACKED TYPE> may be greater than the width of the unpacked type. This allocation is also implementation-dependent.

4. The VAL attribute may be used to specify that a type is to be treated as read-only.

EXAMPLE

```
TYPE byte {0 TO 255};
TYPE read_only DESC OF VAL byte;
TYPE ptr_to_val_int PTR TO VAL byte;

DCL p PROC(dvb read_only) IS
BLOCK
   DCL
      rptr ptr_to_val_int,
      i byte;

   dvb[0] + 1 -> i;      % legal.
   i + 1 -> dvb[0];      % illegal. dvb[0] is read-only.
   PTR i -> rptr;        % legal.
   rptr@ + 2 -> i;       % legal dereference of rptr.
   20 -> rptr@;          % illegal, since rptr points to
                         % a read-only type.
ENDBLOCK;
```

In procedure p, dvb is a descriptor which points to a table the elements of cannot be updated; rptr points to a byte which is cannot be updated.

---

[11] PACK is an unfortunate terminology, since it is used to "pad" the width of a type.

7.3 Structure Overlays

SYNTAX

```
<STRUCT TYPE> ::= STRUCT <DCL LIST> ENDSTRUCT

<DCL LIST> ::= <DCL>
             | <DCL LIST>, <DCL>

<DCL> ::= <ID LIST> <TYPE>
        | <OVLY GROUP>

<OVLY GROUP> ::= OVLY <TYPE> <OVLY BODY> ENDOVLY

<OVLY BODY> ::= <CASE LABEL> <DCL LIST>
              | <OVLY BODY> <CASE LABEL> <DCL LIST>

<CASE LABEL> ::= {<EXPRSN LIST>} :

<EXPRSN LIST> ::= <EXPRSN>
                | <EXPRSN LIST> , <EXPRSN>
```

EXAMPLES

1.
```
TYPE country {canada, usa, uk};

    TYPE phone_record
       STRUCT
          name TABLE[0 TO 20] OF char,
          c    country,  % overlay tag field.
          OVLY country
             {canada, usa}:
                 digits1 TABLE[0 TO 10] OF digit
             {uk}:
                 digits2 TABLE[0 TO 14] OF digit,
                 length {4 TO 15}
          ENDOVLY
       ENDSTRUCT;
```

2.
```
TYPE day_struct
      STRUCT
          OVLY {1 TO 2}
             {1}: p weekday
             {2}: g int
          ENDOVLY
       ENDSTRUCT;
```

3.
```
DCL  s
       STRUCT
          d        {0 TO 500},
```

```
    OVLY {v,w,x,y,z}
       {w}:   e  BOOL,
              f  PTR TO t
       {x,y}: h  weekday
       {z}:
          OVLY BOOL
             {TRUE}:  k  PTR TO j
             {FALSE}: q  index
          ENDOVLY
       {v}:  r  int
    ENDOVLY
ENDSTRUCT;
```

DESCRIPTION

1. Overlays are used within structures so that a given structure element can have multiple types. The reasons why this might be done are:

a. To allow an element within a structure to have different types in different instances of the structure. For example, if m and n are two instances of day_struct in example 2 above, the type of m could be weekday and the type of n could be int.

b. To effect storage savings by having a given instance of the structure hold variables of different types at different times.

c. To suppress type checking implicitly by having one instance of a structure accessed as different types at the same time. An element of an overlay assigned a value of one type would be retrieved as a value of another type.

The difference between situation b and c above can be illustrated by the following.

Assume ds is a variable of type day_struct. ds contains a field p of type weekday overlayed with field g of type int. Situation b occurs in the sequence:

a. assign instance of type weekday to p

b. retrieve p as type weekday

c. assign instance of type int to p

d. retrieve g as type int

Situation c occurs in the sequence:

a. assign instance of type weekday to p

b. retrieve g as type int.

Of the above ways of using overlays b is preferred; c is undesirable as it bypasses the type checking mechanism of the compiler.[12] a is acceptable but of limited value; an appropriately declared AREA and refinements would permit the same space saving, enhance readability, and might allow greater type checking by the compiler.

2. The labels of different overlay alternatives must be constant expressions or tuples of the type stated or defined in <OVLY HEAD>.

3. Every item in a given <OVLY GROUP> must be unique. This is required to allow unique determination of the overlay associated with a reference to a name.

4. Each overlay in an <OVLY GROUP> is allocated independently from all other overlays in the group. This implies that the value of the allocation point (word number and bit offset) at the start of the overlay is saved, and restored for each case label encountered in <OVLY GROUP>.

5. The value of the allocation point, at the end of the overlay, is set to the highest value attained for any overlay in <OVLY GROUP>. Consider the following incomplete example, assuming that the default size of type s1 is less than or equal to 6 bits.

---

[12] Using overlays to bypass type checking is a dangerous programming practice. (It is documented here because it occurs often in existing software.) When type checking must be defeated, CAST is the appropriate mechansim for doing so. CAST is checked by the compiler for various dangers, while using an overlay as above is completely unchecked. As well, using an overlay to defeat type checking hides the intent of the overlay from readers of the code; the intent of a CAST is quite clear.

```
                                % OFFSET SIZE

TYPE s2                         % 0000.0 0002.6
  STRUCT                        %
    a  PACK(6)   s1,            % 0000.0 0000.6
    OVLY {1 TO 3}               %
      {1}: b  PACK(14) s1       % 0001.0 0000.E
      {2}: c  PACK(20) s1       % 0001.0 0001.4 <-- ovly size
      {3}: d  PACK(15) s1       % 0001.0 0000.F
    ENDOVLY,                    %
    r {0 TO 2}                  % 0002.4 0000.2
  ENDSTRUCT;
```

Using PROTEL/DMS standard packing rules, which are based on a 16-bit word, we have filled in the offset and size of each field. The format used here is “words.bits". Each field of the OVLY has the same offset, i.e., they are overlayed. The overlay has size 1.4, the width of the widest overlayed field. Note that field r, immediately following the overlay, is at offset 2.4.

Packing rules are implementation-dependent. In standard DMS packing, a field of a structure begins at the current allocation point if it can fit within the remainder of the current word; otherwise the allocation point jumps to the beginning of the next word. Hence there is a gap of 10 bits between fields a and b, but no gap between the overlay and field r.

6. If more than one <EXPRSN> appears in <EXPRSN LIST> on an overlay label the overlay applies for any of the <EXPRSN>s in <EXPRSN LIST>.

```
TYPE def {d,e,f};

TYPE s3
   STRUCT
      ovly_tf def, % overlay tag field.
      OVLY def
         {d, f}: i int
         {e}:    j weekday
      ENDOVLY
   ENDSTRUCT;
```

Multiple labels on an overlay expression are a convenience to the programmer, since the compiler does not check that the fields are being accessed appropriately. In cases d and f of s3 the same variable i is deemed to exist.

7. An overlay tag field is a field of a structure whose type is the same as the selector type for the overlay groups. In example 1, c is a tag field. The value of a tag field indicates which overlayed field is currently defined. For example, when c = canada, the field digits1 should be referenced, and fields digits2 and length should not be referenced. One should always check the value of the tag field to insure that the correct overlay group is being referenced.

PROTEL does not insist that an OVLY be accompanied by a tag field.

8.0 Scope of Identifiers

The scope of an identifier is the region in a program where the identifier is known and may be referenced. The scope of an identifier, in general, depends on where the identifier is declared. An identifier in this section may be a user-defined type, variable, constant, label or formal parameter.

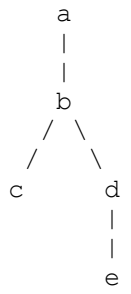
8.1 Scope Rules

1. Within a section, identifiers declared outside of a procedure are global and are known from the point of their declaration to the end of the section. The scope of these global identifiers extends into procedure bodies unless the identifier is redeclared inside the procedure. In such cases, the more recent declaration applies.[13]

Within a module, if section x imbeds section y, the global identifiers in y are known throughout x. The same identifier may not be declared globally in both x and y. In more complex imbedding situations the identifiers declared along any imbedding path must be unique.

EXAMPLE

```
INTERFACE a;
 .
 .
SECTION  b  OF  a;
 .
 .
SECTION  c  OF  b;
 .
 .
SECTION  d  OF  b;
 .
 .
SECTION  e  OF  d;
```



In the above sequence of sections, identifiers must be unique throughout paths {a,b,c}, {a,b,d,e}.

2. All identifiers declared within a procedure are local to that procedure: they are known within that procedure and, if they are not variables, within any contained procedure. If an identifier declared outside a procedure is redeclared within the procedure, the inner declaration supercedes the outer one.

3. A procedure's formal parameters and local variables are strictly local to the procedure. By "strictly local" we mean they are not visible inside the body of any nested procedure.[14]

---

[13] If the identifier is a global type, its scope is the entire section, regardless of its point of declaration. When a type identifier is referenced before its declaration, we call this an instance of a "forward type." (This is quite different from FORWARD procedures.) The use of forward types is discouraged, and as a good programming practice types should be declared before use.

[14] This is one reason why nested procedures are considered of little use in PROTEL. Nested procedures are being removed from the language, and are not allowed in PROTEL/68K.

EXAMPLE

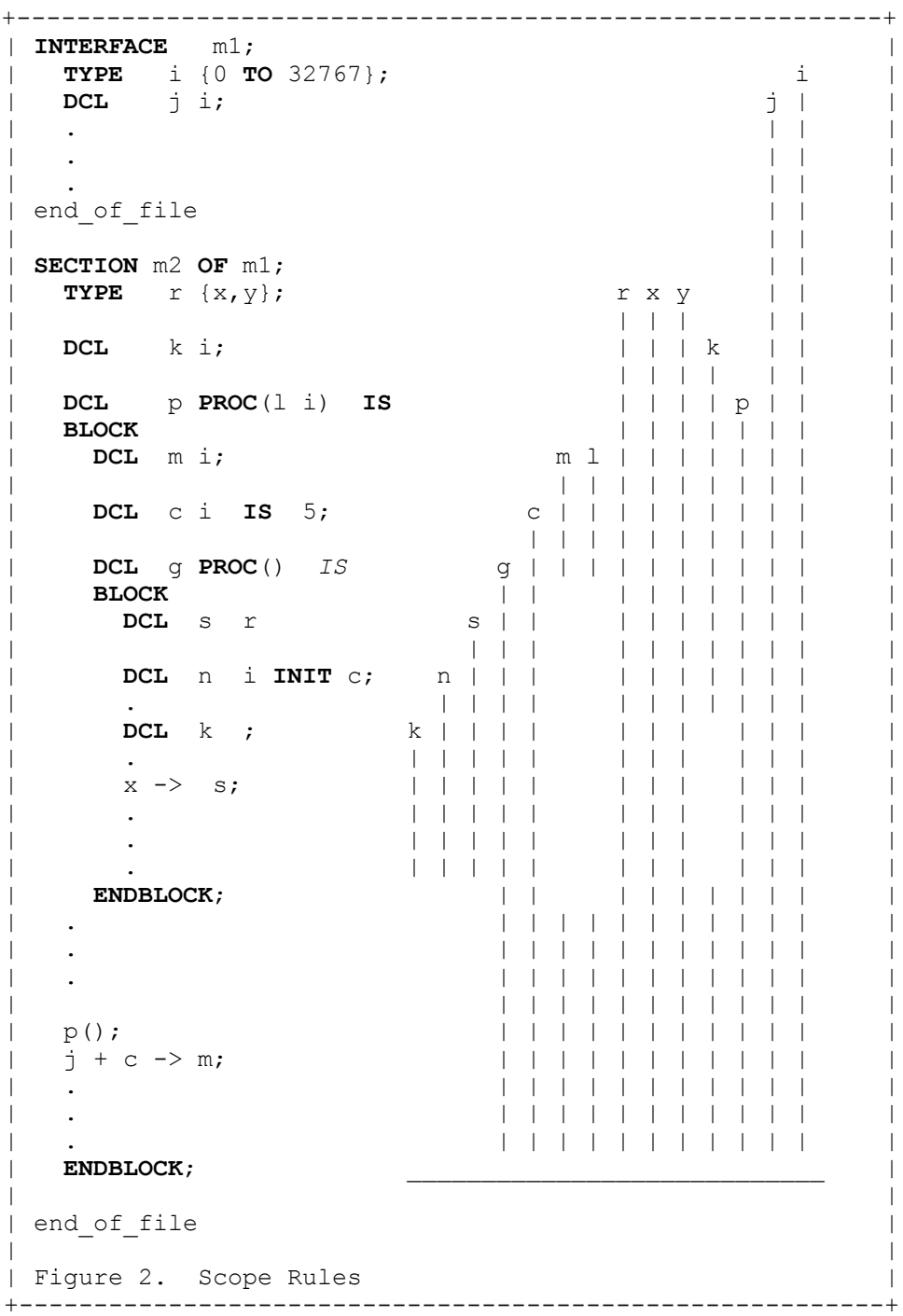


Figure 2 illustrates the scope rules 1, 2, and 3. The declared identifiers and the regions where they are known are shown on the right. Note especially the "hole in the scope" in procedure g for the local variable m and the formal parameter l.

4. Across modules an identifier may be reused, however this makes references to the identifier ambiguous. Suppose modules q and r both declare global identifier zed in their

respective interfaces. Suppose also that module s USES q and r. If a reference to zed occurs in s, it could mean zed in q or zed in r. Such ambiguity is resolved by a module qualification on the reference. Module qualification of an identifier is the prefixing of the identifier with a module name and two colons. In module q, zed would have to be qualified as "q::zed" or "r::zed".

8.2 Local Variable Block

SYNTAX

```
<BLOCK> ::= BLOCK <OPT STMT SEQ> ENDBLOCK <OPT END LABEL>

<OPT STMT SEQ> ::= <OPT DCL STMT SEQ> <OPT EXEC STMT SEQ>

<OPT DCL STMT SEQ> ::=
                     | <OPT DCL STMT SEQ> <DCL STMT> ;

<OPT EXEC STMT SEQ> ::=
                      | <OPT EXEC STMT SEQ> <STMT> ;

<DCL STMT> ::= <TYPE DCL STMT>
             | <DATA DCL STMT>
             | <WITH DCL STMT>
             | <BIND DCL STMT>

<OPT END LABEL> ::=
                  | <ID>
```

EXAMPLES

1.
```
DCL  p  PROC(x int) IS
     BLOCK
        IF x > 0 THEN
           F(x);
        ENDIF;
     ENDBLOCK;
```

2.
```
DCL  g  PROC()  IS
     BLOCK
        DCL  i  int;

        0 -> i;
     l: BLOCK
           DCL  j  int;
           .
           .            % j is known only within block l.
           j -> i;      % Note that the variable i is
        ENDBLOCK l;     % known in all blocks contained

                        % in g that are not procedure blocks.
        BLOCK
           DCL  k  int;
           .
           .
           .
           i + k -> i;
        ENDBLOCK;
     ENDBLOCK;
```

DESCRIPTION

1. Blocks are used to delineate procedure bodies, to support declarations local to a region of a procedure and to designate a group of statements from which an exit may be made using the EXIT statement.

2. When a procedure is entered, storage is allocated for the variables declared in its outermost block. Block variables are assigned storage so that disjoint blocks share the same space. This can reduce significantly the total amount of storage required for the procedure.

3. Identifiers declared in a block are known throughout the block and within any contained block. If an identifier, declared in an outer block, is redeclared in an inner block the inner declaration supercedes the outer throughout the duration of the inner block. All identifiers declared in a block are unknown outside the block.

4. Blocks may be labelled for the purpose of program documentation or debugging or to name the block for reference by a contained EXIT statement. The EXIT statement is described in "EXIT Statement".

5. The following statements contain implicit blocks (i.e., they may contain declarations without the BLOCK and ENDBLOCK keywords appearing).

- The THEN part of an IF statement;
- The ELSE part of an IF statement;
- Statements between DO and ENDDO;
- The OUT clause of a CASE statement.

NOTE: A BLOCK ENDBLOCK pair is still required when defining procedure constants of more than one statement and in the in-range cases of a CASE statement.

9.0 Executable Statements

SYNTAX

```
<STMT> ::= <UNLABELLED STMT>
         | <LABEL> <UNLABELLED STMT>

<LABEL> ::= <ID> :

<UNLABELLED STMT> ::= <BLOCK>
                    | <EXPRSN>
                    | <RETURN STMT>
                    | <EXIT STMT>
                    | <IF STMT>
                    | <CASE STMT>
                    | <SELECT STMT>
                    | <ITERATIVE STMT>
```

DESCRIPTION

1. The preceding sections have described the declarative statements used to define the data accessible to a program. This section describes the executable statements which operate on this data.

2. The executable statements can be categorized as expressions or control statements. Expressions are used to create new data values from existing data items. Control statements are used to modify the normal sequential order of execution of statements. In the above syntax, all the <UNLABELLED STMTS> except <EXPRSN> and <BLOCK> are control statements.

3. An <EXPRSN> is a <STMT> if and only if it consists of a single procedure call or is an assignment statement (contains an assignment operator (->) as its last evaluated operator).=

4. The basic components of an <EXPRSN> are operands, operators and brackets. Operands in an expression may be constants, storage references or procedure calls. These components are described in the next three sections followed by a description of the overall expression structure.

5. The BLOCK, CASE, SELECT, IF and ITERATIVE statements can enclose one or more statements between the starting and terminating keywords (e.g., CASE, ENDCASE). When one of these five statements is labelled, the terminating keyword may be followed by a label (<OPT END LABEL>) which must match the outermost label on the initial keyword: e.g.,

```
x: BLOCK
  .
  .
  .
ENDBLOCK x;
```

   When a block is the body of a procedure constant, the label on the block is assumed to be the procedure name.

9.1 Expressions

9.1.1 Components of an Expression

9.1.1.1 Simple Constants

SYNTAX

```
<CONSTANT> ::= <NUMERIC CONSTANT>
             | <BOOL CONSTANT>
             | <TUPLE>
             | <PTR CONSTANT>

<NUMERIC CONSTANT> ::= <DECIMAL NUMBER>
                     | <HEX NUMBER>

<BOOL CONSTANT> ::= TRUE
                  | FALSE

<PTR CONSTANT> ::= NIL

<TUPLE> ::= <OPT REPETITION FACTOR> {<OPT TUPLE BODY LIST>}
          | <OPT REPETITION FACTOR> <STRING>
```

DESCRIPTION

1. Self defining constants include scalar and aggregate constants as described in "Simple Constant Expressions" through "Character Strings", the boolean constants TRUE and FALSE, and the pointer constant NIL. The constant NIL may also be used to initialize descriptors or procedure variables to a null value.

2. Symbolic constants, created by a declaration with an “IS value” clause, are described under the category of storage references.

9.1.1.2 Storage References

SYNTAX

```
<STORAGE REF> ::= <ID>
                | <SELECTION>
                | <DEREFERENCE>
                | <INDEXING>
                | <MULTIPLE>
                | <PROC CALL>
                | CAST <EXPRSN> AS <TYPE>

  <SELECTION> ::= <STORAGE REF> . <ID>

  <DEREFERENCE> ::= <STORAGE REF> @

  <INDEXING> ::= <STORAGE REF> [ <EXPRSN> ]

  <MULTIPLE> ::= <STORAGE REF> [ <OPT SUBRANGE> ]

  <OPT SUBRANGE> ::=
                  | <EXPRSN> TO <EXPRSN>
```

EXAMPLES

Assume the following declarations:

```
TYPE integer {-32768  TO  32767};

DCL  size   integer IS  60;

DCL  index, j {1 TO 100};

TYPE tab1   TABLE[1 TO 10] OF integer;

DCL  ttab1  tab1;

DCL  ptab1  PTR  TO  tab1;

TYPE struct1
    STRUCT
       b   BOOL,
       p   PTR TO struct1,
       i   integer,
       t   TABLE[0 TO 7] OF BOOL
    ENDSTRUCT;

DCL s1  struct1;

DCL s2
    STRUCT
```

```
        i    integer,
        s    struct1,
        b    BOOL
    ENDSTRUCT;

DCL s1tab  TABLE[1 TO 4] OF struct1;

DCL desc1 DESC OF integer;

DCL tabdes TABLE[1 TO 10] OF DESC OF integer;

DCL matrix TABLE[1 TO 5] OF TABLE [1 TO 20] OF integer;
```

The following are examples of valid storage references:

1. `ttab1[j]` % jth element of ttab1, an integer.
2. `s1.b` % element b of s1, a boolean.
3. `ptab1@[j]` % If ptab1 points to tab1, this is same % item as tab1[j]
4. `s1.t[0];` % first element of table t in s1, a boolean
5. `s1tab[3]` % an instance of struct1
6. `s1tab[3].p@` % another instance of struct1 linked to the % first by pointer p.
7. `s1tab[3].p@.p@.t[j]` % jth boolean in table t in 3rd instance of % struct1 in the 3rd instance of s1tab.
8. `desc1` % 3 word descriptor
9. `desc1[j]` % jth element of table pointed to by % descriptor, an integer
10. `desc1[]` % entire table pointed to by descriptor, a % table of integers
11. `desc1[3 TO 7]` % table of 5 entries, pointed to by desc1, % extending from element 3 % to element 7.
12. `ttab1` % illegal. (The DMS implementation accepts % this as a storage ref in certain cases, % meaning ttab1[], but the semantics are % not quite the same as ttab1[] and the % bracketed form is prefered.)
13. `ttab1[]` % entire table ttab1.

14. `tabdes[3][j]` `% jth integer of table pointed to`
`% by element 3's descriptor in tabdes.`

15. `matrix [5][20]` `% last element in matrix, an integer.`

16. **`CAST`** `s1.t` **`AS`** `integer % table of bool t is treated as an integer`

DESCRIPTION

1. Storage references are used to represent either variables or symbolic constants.

2. The <DEREFERENCE> form of a storage reference is used to denote the item pointed to by a pointer variable. The item is an instance of the type, <TYPE>, as defined in the pointer declaration PTR TO <TYPE>. The <STORAGE REF> preceding the dereferencing operator, @, must be a pointer variable or an expression evaluating to a pointer value.

   The special pointer value NIL is used to indicate that a pointer does not point to any valid instance of <TYPE>. It is illegal to dereference a NIL pointer.

3. The <ID> form of a storage reference represents a symbolic constant or a variable. The effect of such a reference is to yield the value associated with <ID>.

4. The <SELECTION> form of a storage reference is used to select an element of a structure. A reference of the form <STORAGE REF>.<ID> denotes the element <ID> of the structure variable <STORAGE REF>. If a structure is nested in other structures, an element of that structure must be qualified by the name of all nesting structures. For example, to access the element p in the structure struct1 within the structure variable s2, the reference

   ```
   s2.s.p
   ```

   would be used.

5. The <INDEXING> form of a storage reference is used to select an element of a table. The storage reference preceding the bracketed subscript expression must be either a table or a descriptor. The type of <EXPRSN> must match the type given for the table or descriptor bounds. This type must be a numeric range or a symbolic range.

   If the value of <EXPRSN> does not lie within the bounds of the subscript type, an error message will be generated. If the elements of a table (or descriptor) are themselves tables, (or descriptors), two or more bracketed subscript expressions are used to identify a particular element.

   NOTE: The same form is used to reference a table element whether the table is accessed directly or via a descriptor.

6. The <MULTIPLE> form of a storage reference is used to indicate that a reference denotes more than one element of a table or descriptor. If <OPT SUBRANGE> is present the subportion of the table from the element indexed by the first <EXPRSN> up to and including the element indexed by the second <EXPRESN> is assumed. The value of the

first <EXPRSN> must be less than or equal to that of the second. If there is no <OPT SUBRANGE> present, the whole table is assumed.

NOTE: A whole table cannot be referenced merely by its name, square brackets are required to indicate the intent. This rule ensures that references to tables either directly or by way of descriptors are handled in a consistent manner. In the case of a descriptor, the use of just the name denotes the descriptor itself and not the table pointed to by the descriptor.

7. The CAST form of a storage reference is used to circumvent type checking. The compiler treats <EXPRSN> as being of the specified <TYPE>. Roughly speaking, a CAST is considered safe if either the number of bits occupied by the result of the expression is equal to that of the type or both occupy less than a word of storage. The use of CAST should be avoided unless necessary.

8. A CAST applies to only one reference; it does not permanently change the type of the expression. For example

```
DCL i integer,
    b, c BOOL;

(CAST i AS BOOL & b) | (i & c) -> c;
```

will result in an error at (i & c) since i is an integer.

9. A constant can sometimes be a storage reference. This depends upon the size of the constant and is implementation-dependent. In PROTEL/DMS, a constant which has a bit width less than or equal to 48 bits cannot be used as a storage reference.

Recalling that the PTR operator can be applied only to storage references, consider the following possible operands for PTR. In both the DMS and IBM/370 implementations, neither size nor c are storage references, while both i and d are.

EXAMPLES

```
TYPE int {-32768 TO 32767};

DCL
   size int IS 123,
   i    int,
   c char IS 'Z',
   d DESC OF char IS 'Z';
```

1. `PTR size` % illegal, size is not a % storage reference.
2. `PTR i` % okay. A variable is always % a storage reference.
3. `PTR c` % illegal. c is not a % storage reference.
4. `PTR d` % okay. A desc is always % allocated storage.

9.1.1.3 Procedure Calls

SYNTAX

```
<PROC CALL> ::= <STORAGE REF> ( <OPT ACTUAL PARM LIST> )

  <OPT ACTUAL PARM LIST> ::=
                            | <ACTUAL PARM LIST>

  <ACTUAL PARM LIST> ::= <EXPRSN>
                       | <ACTUAL PARM LIST>, <EXPRSN>
```

EXAMPLES

1.
```
find(idle_trunk);     % invoke procedure "find"
                          % with actual parameter idle_trunk.
```

2.
```
start();               % invoke procedure "start"
                          % with no parameters
```

3.
```
max(max(a,b),c) -> x; % function "max" returns
                          % the larger of its two parameters
                          % This statement assigns the
                          % largest of a, b and c to x.
```

4.
```
proc_tab[j](parm1);    % proc_tab is a table of procedure
                          % variables from which the jth
                          % variable is selected and the
                          % procedure it references is
                          % invoked with parm1 as the actual
                          % parameter.
```

5.
```
proc_struct.proc1(5); % the procedure variable proc1 in
                          % proc_struct is selected and the
                          % procedure it references is invoked
                          % with actual parameter 5.
```

6.
```
proc_returning_function(third)(a,b);
                          % proc_returning_function is
                          % a function which returns
                          % a procedure variable. The
                          % procedure referenced by
                          % this variable is invoked
                          % with the actual parameters
                          % a and b.
```

DESCRIPTION

1. A <PROC CALL> is used either by itself or as a component of an expression (i.e., a function). When used as a function, the procedure must have been declared with a RETURNS clause indicating the type of the value to be returned by the function. When used by itself, the procedure in <PROC CALL> must have been declared without the RETURNS clause.

2. The <STORAGE REF> appearing in <PROC CALL> is usually an <ID> representing a procedure constant but it may correspond to any storage reference which yields a procedure constant as a value (see examples 5, 6 and 7 above).

3. The number of actual parameters must be the same as the number of formal parameters in the procedure declaration. The actual parameter for a REF or UPDATES formal parameter must be a storage reference, which may be any expression which evaluates to a storage reference. (A precedure call which returns a storage reference is a storage reference.)

   At the time of procedure invocation, each actual parameter is evaluated and an implicit assignment is made from the actual parameter to the corresponding formal parameter in the procedure body. If the formal parameter is not declared either REF or UPDATES, the value (content) of the actual parameter is assigned.

   If the formal parameter has the REF or UPDATES attribute, the address of the actual parameter is assigned to the formal parameter. However, from inside the procedure body, it is referenced as though it were the object itself, not as a pointer. The pointer dereferencing is implicit in the designation of REF or UPDATES as the parameter passing mechanism.

   EXAMPLE

```
DCL p PROC (t TABLE[0 TO 10] OF int) IS FORWARD;
DCL p PROC (UPDATES t TABLE[0 TO 10] OF int) IS FORWARD;
```

   Whether procedure p is declared by either of the procedure declarations above, element i of table t would be referenced as t[0]. It would not be referenced as t@[0] given the latter declaration.

4. Note that a <PROC CALL> of a procedure without parameters must appear with an empty parameter list (i.e., p()). This is necessary to distinguish between assignment of procedure return values and procedure variable assignment.

   EXAMPLE

   Assume p1 is a procedure constant and p2 is a procedure variable.

```
p1 -> p2;       % assigns procedure p1 to variable p2.

p1() -> p2();    % assigns the result of p1 to the
                 % result of p2. p2 must return a
                 % storage reference.
```

9.1.2 STRUCTURE OF AN EXPRESSION

SYNTAX

```
<EXPRSN> ::= <SECONDARY>
           | <EXPRSN> <OPERATOR> <SECONDARY>

  <SECONDARY> ::= <PRIMARY>
                | <OPERATOR> <SECONDARY>

  <PRIMARY> ::= <CONSTANT>
              | <STORAGE REF>
              | <PROC CALL>
              | ( <EXPRSN> )

  <OPERATOR> ::= ->
               | +
               | -
               | *
               | /
               | MOD
               | &
               | "|"
               | !
               | ^
               | =
               | ^=
               | <
               | >
               | <=
               | >=
               | <<
               | >>
               | PTR
               | DESC
               | UPB
               | TDSIZE
               | INCL
               | NOTINCL
```

EXAMPLES: Assume the following declarations have been made:

```
TYPE  int {0 TO 32767};
TYPE  ti  TABLE[0 TO 9] OF BOOL;
DCL   i, j, k int;
DCL   b   BOOL;
DCL   t   TABLE[1 TO 5] OF int;
DCL   tb  ti;
DCL   d   DESC  OF int;
```

```
DCL   p   PTR TO int;
DCL   pr  PROC(a ti);
DCL   s   SET {x, y, z};
DCL   f   PROC(c int) RETURNS int;
```

The following are valid expressions (except example 9):

1. `5 -> i`

2. `i + 1 -> i`

3.
```
-(3 * (j - 4)) -> i + 1 -> j;
      j = (i + 1) -> b;      % b now has value TRUE.
```

4. `(i < 5) | ((j=0)) & (k << 4 < 0)`

5. `(i + 1) -> k`

6.
```
5 MOD 2 = 1            % boolean value = TRUE
```

7.
```
{x,z} -> s;            % Assignment to s sets x and z
      {y} INCL s           % and clears y. The second
                           % expression has value FALSE.
```

8.
```
PTR  p@ = p            % value is TRUE. The dereference
                           % operator @ is evaluated before
                           % PTR.
```

9.
```
PTR i@ = i             % illegal. i may not be
                           % de-referenced.
```

10.
```
NIL -> p1@.e -> p1     % e is a pointer element in the
                           % structure instance pointed to by
                           % p1. Both e and p1 are set to NIL.
```

11.
```
NIL -> p1 -> p1@.e     % this will trap at run-time, since
                           % p1 is dereferenced after it is set
                           % to NIL.
```

12. `UPB d + j -> i`

13.
```
i << 5                 % equivalent to i*32.
```

14.
```
i >> 3                 % equivalent to i/8 if i >= 0.
```

15.
```
-32 >> 3               % N.B. not equal to -4 since zeros
                           % are shifted into sign position.
```

16.
```
UPB t -> k             % k now has the value 5.
```

17.
```
pr(tb[])               % invocation of procedure pr with
```

```
                                % table tb as a parameter.

18. i * (i + 1 -> i)        % if i has initial value 3 then the
                               % value of the expression is 12 and
                               % the final value of i is 4.

19. TDSIZE t                % returns 5.

20. UPB tb                  % returns 9.

21. TDSIZE tb               % returns 10.
```

DESCRIPTION

1. Expressions are sequences of constants, storage references, procedure calls and operators used to compute a value or invoke a procedure. An expression consisting of a procedure call or having the assignment operator (->) as its last evaluated operator can be treated as a single statement.

2. Expressions are evaluated from left to right with all arithmetic and logical binary (2 operand) operators having the same precedence.[15] The unary (one operand) operators (-, +, ^, PTR, DESC, UPB, TDSIZE) are applied to the operand immediately to their right and have higher precedence than the binary operators. The normal order of evaluation may be modified, as in ordinary algebra, by the use of parentheses.

---

[15] A storage reference in an expression is always the longest possible sequence of symbols which can be interpreted as a storage reference. For example, in the expression

```
PTR p@ -> i;
```

the operand of PTR is <STORAGE REF>. The longest storage reference is p@, hence the @ (dereference) operator is performed before PTR. As second example, note that the boolean expression

```
b | t[7].f2@
```

requires no brackets to be evaluated correctly. If this were not the case, the expression

```
t[7].f1 + t[6].f1 -> i;
```

would have to be written as

```
(t[7].f1) + (t[6].f1) -> i;
```

3. The assignment operator (->) stores the value of its left operand in the <STORAGE REF> on its right. This <STORAGE REF> must be alterable, i.e., it must not have the VAL attribute as part of its type and must not be a REF parameter. The value resulting from an assignment operation is the value of the left operand. Caution should be exercised when the assignment operation is not the final operation in evaluating an expression. For example, the result of:

```
i * ( i + 1 -> i)
```

is not the same as:

```
(i + 1 -> i) * i
```

If i has initial value 3, the first expression has a result of 12 but the second expression has a result of 16. This occurs because the value of i in the first case is loaded before the expression in parentheses is evaluated and is therefore unaffected by the assignment.

4. The arithmetic operators (+, -, *, /, MOD) are used for standard twos complement arithmetic operations on instances of type numeric range. Note that signed numbers may not be stored in variables which occupy less than a full word.

The / operator performs integer division, returning the integral portion of the result. The MOD operator performs the modulus operation; it returns the integer remainder of the division of its left operand by its right operand. PROTEL's definition of mod is

```
M MOD N  =  M - (M / N * N)
```

which varies from the mathemtical definition. In particular, it is possible with PROTEL's definition, to get a negative result, when the operands are negative.

5. The function of the boolean operators (&,|,!,^) depends on the type of their operands. When both operands are of type BOOL, they perform the logical operations inclusive OR (|), AND (&), exclusive OR (!), and negation (^). When both operands are of type numeric range or SET they perform the bitwise OR, AND, exclusive OR and complement functions (i.e., set union, intersection, exclusive OR and complement).

6. The equality operators (=,^=) test the equality or inequality of their two operands and return a boolean value. The operands may be of any compatible type (see "Type Compatibility" on page 93).

7. The shift operators << and >> perform logical left and right shift operations on operands of type numeric range. The left operand is the item to be shifted and the right operand is the shift amount. The shift amount should be in the range 0 up to the number of bits in a word.

Bits shifted out are lost; bits shifted in are zero.

8. The set inclusion operators (INCL, NOTINCL) test whether the set on their left is a subset of the set on the right. Both operators return a boolean value. The operators INCL and NOTINCL read "is included in" and “is not included in", respectively. For example, given the declaration

```
DCL a SET {x,y,z} INIT {x,z};
```

then examples of TRUE expressions are:

```
{x} INCL  a
{z,x} INCL a      % note elements do not have to appear
                  % in the same order as in set definition.
{ } INCL  a
{x} INCL {x}      % a subset does not need to be a
                  % "proper" subset.
```

and examples of FALSE expressions are:

```
{x} NOTINCL a
{y} INCL a
{ } NOTINCL a
{x,y,z} INCL a
```

9. The relational operators (<,>,<=,>=) compare (algebraically) the relative magnitude of their two operands and yield a boolean value as a result. The operands must both be of type numeric range.

10. The unary operator UPB when applied to a descriptor or table variable or constant yields the value of the upper bound of the table or descriptor. The operator TDSIZE, when similarly applied, returns the number of items in the table or descriptor. These operators may not be applied to a multiple (i.e., "UPB t" is legal, "UPB t[]" is not legal). The TDSIZE operator returns an unsigned numeric value. The UPB operator returns a result of the same type as the index type for the table or descriptor to which it is applied. Given declarations:

```
TYPE office_type {local, toll, combined};
DCL  office TABLE [office_type] OF DESC OF char;
```

"TDSIZE office" is unsigned value 3, while "UPB office" is value "combined" of type office_type.

11. The unary operator PTR when applied to a <STORAGE REF> yields the address of <STORAGE REF>. The type of the expression "PTR TO <STORAGE REF>" is is PTR TO <TYPE> where <TYPE> is the type of the <STORAGE REF>.

PTR may not be applied to any variable which does not begin on an addressable boundary in storage.

If t is a table variable, it must not be a multiple, e.g., neither "PTR t[]" nor "PTR t[0 TO x]" are allowed.[16]

---

[16] The IBM/370 implementation differs in this respect: "PTR t[]" is allowed, but "PTR t" is not. The multiple may not have a subrange expression, e.g., "PTR t[0 TO 5]" is not allowed.

In the following example, the two pointer expressions yield the same the value but differ in type. They both yield the address of the first element of t.

EXAMPLE

```
TYPE tabl TABLE[a TO b] OF some_type;
DCL t tabl;

PTR t        % expression type is PTR TO tabl.
PTR t[0]     % expression type is PTR TO some_type.
```

12. The DESC unary operator when applied to a table yields a descriptor of the table. The table variable must be a multiple with or without a subrange. For example, the following are valid DESC operators:

```
DESC t[]         % upper bound = (number of elements in
                 %               table) - 1
DESC t[3 TO 7]   % upper bound = 4
```

The type of the descriptor created by DESC is DESC OF <TYPE> where <TYPE> is the type of the table elements.

9.2 RETURN Statement

SYNTAX

```
<RETURN STMT> ::= RETURN
                | RETURN <EXPRSN>
```

EXAMPLES

1. `RETURN 0;`
2. `RETURN;`
3. `RETURN z * (i + 5);`

DESCRIPTION

1. In the RETURN statement <EXPRSN> appears if and only if the procedure containing the statement was defined with the RETURNS clause.

2. The RETURN statement in a procedure causes control to be returned to the point of invocation of the procedure. If a procedure is nested within another, the RETURN applies to the innermost containing procedure.

3. Every procedure invoked as a function must contain at least one RETURN <EXPRSN> statement. The value of <EXPRSN> is returned as the value of the function.

4. In a procedure defined without the RETURNS clause, a RETURN statement is always implied as the last statement of the procedure body; generally, this will be ENDBLOCK.

9.3 IF Statement

SYNTAX

```
<IF STMT> ::= IF <EXPRSN> THEN <OPT STMT SEQ> <OPT ELSE PART>
                ENDIF <OPT LABEL>

<OPT STMT SEQ> ::=
                 | <OPT DCL STMT SEQ> <OPT EXEC STMT SEQ>

<OPT DCL STMT SEQ> ::=
                      | <OPT DCL STMT SEQ> <DCL STMT> ;

<OPT EXEC STMT SEQ> ::=
                       | <OPT EXEC STMT SEQ> <STMT> ;

<OPT ELSE PART> ::=
                  | ELSE <OPT STMT SEQ>
```

EXAMPLES

1.
```
IF (a > 5) | (stat = busy) THEN
                           % note brackets on second clause
                           % required because of left to
                           % right evaluation.
      a - 1 -> a;
      sub(a);       % procedure call
   ENDIF;
```

2.
```
lab1: IF ^ b THEN
            0 -> x;
         ELSE
            x + 1 -> x;
            ok -> flag;
         ENDIF lab1;    % label lab1 must match label on IF.
```

DESCRIPTION

1. The IF statement is used to conditionally execute a sequence of statements. The <EXPRSN> following IF must yield a value of type BOOL. If <EXPRSN> is TRUE, the sequence of statements from THEN up to ELSE (if present) or ENDIF (if there is no ELSE) is executed.

If <EXPRSN> is FALSE, and an ELSE part is present, the sequence of statements between ELSE and ENDIF are executed. If <EXPRSN> is false and no ELSE part is present, execution continues with the statement following ENDIF. Note that a statement in the above statement sequences may itself be an IF statement. For example:

```
IF b1 THEN
   a();  % procedure call with no parameters.
   IF b2 THEN
```

```
      % do nothing
   ELSE
      c();
   ENDIF;
   d();
ELSE
   e();
ENDIF;
f();
```

For the four possible combinations of the values of b1 and b2 the sequences of procedure calls would be:

b1	b2	calling sequence
FALSE	FALSE	e,f
FALSE	TRUE	e,f
TRUE	FALSE	a,c,d,f
TRUE	TRUE	a,d,f.

9.4 CASE Statement

SYNTAX

```
<CASE STMT> ::= CASE <EXPRSN> IN <CASE BODY> ENDCASE
                <OPT END LABEL>

<CASE BODY> ::= <IN RANGE CASES> <OPT OUT CASE>

<IN RANGE CASES> ::= <CASE LABEL>
                   | <IN RANGE CASES><CASE LABEL>
                   | <IN RANGE CASES> <STMT> ;

<CASE LABEL> ::= <TUPLE> :

<OPT OUT CASE> ::=
                 | OUT <OPT STMT SEQ>
```

EXAMPLES

1.
```
CASE  n  IN           % n is 1st dialed digit
   {0}: TRUE -> operator;
   {1}: TRUE -> ddd_call; set_area();
   {4}: TRUE -> assist; check_11();
   OUT
ENDCASE;
```

2.
```
CASE 2 + (i - 1) IN
   {2,4,8}: 2 -> base;
   {3,9}  : 3 -> base;
   {5,6,7}: 0 -> base;
   OUT     -1 -> base;
ENDCASE;
```

3.
```
CASE status IN     % assume type of status is symbolic
                   % range {busy,blocked,ready,done}
   {done}:  % do nothing
   {ready}: initiated();
            count = 1 -> count;
   {blocked,busy}: wait();
ENDCASE;
```

4.
```
CASE char IN  % assume type of char is
              % numeric range {0 TO 255}
   {' '}: blank -> token;
   '$_' : special -> token;
   '012': digit -> token;
   {'.'}: dot -> token;
```

```
   {'a','b','c','d'}: letter -> token;
   OUT    other -> token;
ENDCASE;
```

5.

```
CASE s IN  % assume type of s is SET {a,b,c,d}
   {{a,b}}  : TRUE  -> valid;  2 -> count;
   {{c},{d}}: TRUE  -> valid;  1 -> count;
   {{ }}    : FALSE -> valid;  0 -> count;
   {{a,b,c,d},{a,b,c}}: TRUE -> valid; 4-> count;
   OUT        FALSE -> valid; 99 -> count;
ENDCASE;
```

6.

```
e1: CASE i IN
  {0}: a();
  {1,2}:
     CASE j IN
        {0}:  b -> c;
        {1}:  c -> d;
        OUT
     ENDCASE;
  {3}: e();
ENDCASE;
```

DESCRIPTION

1. The CASE statement is used to selectively transfer control to one group of statements in a collection of such groups. Each group of statements in the collection is labelled with a tuple containing one or more of the values in the range of the <EXPRSN> following the keyword CASE. When the CASE statement is executed, <EXPRSN> is evaluated and control is transferred to the group of statements for which a label value matches the value of <EXPRSN>. If no match is found, control is transferred either to the group of statements following OUT or, if OUT is omitted, to the statement following the keyword ENDCASE.[17] When the last statement in a group has been executed, control passes to the statement following the keyword ENDCASE.

A statement group extends from a case label to the next case label or the ENDCASE keyword.

[17] In PROTEL/370 it is an error if the value if the value of <EXPRSN> fails to match any selector and the OUT clause is omitted.

2. The value of <EXPRSN> after the keyword CASE must be of type numeric range, symbolic range, set or BOOL.

3. Each value in a case label must be a constant or a constant expression of the same type as the <EXPRSN> after the CASE keyword. Note that SET constants are tuples. Thus labels of type SET will require nested brackets; i.e.:

```
{{a,b}}:     % is the label for the set constant {a,b}.
{{a},{b}}:   % consists of two set labels {a} and {b}.
```

4. If a case label contains more than one value then the corresponding statement group will be executed if the value of the case expression matches any value in the label list. Note that the label:

```
{a,b}:
```

is <u>not</u> equivalent to:

```
{a}:{b}:
```

In the latter case {a} is associated with a null statement group and not with the group for label {b}.

5. A case label of the form:

```
'ABC':
```

can be used with a case expression of type numeric range since this string is equivalent to the tuple

```
{'A','B','C'}:
```

i.e., the statement group with this label will be selected when the value of the case expression is 'A', 'B' or 'C'.[18]

---

[18] This is not a label for the case where the value is the three-character string 'ABC'. In view of paragraph 2 above, a string is not a legal <EXPRSN> expression.

9.5 SELECT Statement

SYNTAX

```
<SELECT STMT> ::= SELECT <EXPRSN> IN <CASE BODY> ENDSELECT
                  <OPT END LABEL>
```

EXAMPLE

```
SELECT n IN
  {0}: TRUE -> operator;
  {1}: TRUE -> ddd_call; set_area();
  {4}: TRUE -> assist;  check_11();
  OUT
ENDSELECT;
```

DESCRIPTION

1. The syntax of the SELECT statement is similar to that of the CASE statement with the exception that SELECT and ENDSELECT are substituted for CASE and ENDCASE. Semantically, SELECT and CASE perform the same action. The difference is in operational efficiency: CASE is usually faster than SELECT but may require more space. The CASE construct performs an indexed branch and requires the same amount of time regardless of the value of the case expression. SELECT compares the value of the expression with all labels in turn and branches when a match occurs. The time required is dependent on the position of the matching label amongst the alternatives. On the other hand, the index table used for CASE has a size proportional to the range of label values; whereas SELECT requires an amount of space proportional to the number of labels.

9.6 EXIT Statement

SYNTAX

```
<EXIT STMT> ::= EXIT <ID>
```

EXAMPLES

1.

```
e: DO    % infinite loop
  read_record(buffer, end_of_file);
  IF end_of_file
     THEN EXIT e;
  ENDIF;
  process(buffer);
 ENDDO e;
```

2.

```
a:
```

```
BLOCK
   :
   :
   :
   b:
    BLOCK
      :
      :
      :
          EXIT a;      % can exit out of any enclosing
      :                % block
      :
    ENDBLOCK b;
    :
    :
    :
ENDBLOCK a;
```

3.

```
queueing:
   IF q_head ^= NIL THEN
      q_head -> q_ptr;
      WHILE q_ptr ^= NIL DO
         CASE q_ptr@.status IN
            {running}: interrupt_and_requeue(q_ptr);
            {blocked}:
            OUT
               IF q_ptr@.process_id = uninitialised THEN
                  % queue is corrupt, cannot continue.
                  EXIT queueing;
               ENDIF;
         ENDCASE;
         check_priority(q_ptr);
         release_resources(q_ptr);
         q_ptr@.next -> q_ptr;
      ENDDO;
      sleep(sched_sleep_time);
      check_clock_ticks();
    ENDIF queueing;
```

DESCRIPTION

1. The EXIT statement is used to branch out of a sequence of statements within a BLOCK, DO, CASE, SELECT, or IF statement.

2. The <ID> following EXIT must match a label on a BLOCK, DO, CASE, SELECT, or IF statement. When EXIT is executed it transfers control to the statement following the terminating keyword (ENDBLOCK, ENDDO, ENDCASE, ENDSELECT, ENDIF) of the block.

3. An EXIT statement may only appear within a labelled BLOCK, DO, CASE, SELECT, or IF statement.

4. An EXIT statement may not be used in place of a RETURN to leave a procedure.
5. An EXITed block must have an end label.

9.7 Iterative Statement

SYNTAX

```
<ITERATIVE STMT> ::= <LOOP CONTROL> DO <OPT STMT SEQ> ENDDO
                     <OPT END LABEL>

<LOOP CONTROL> ::= <OPT FOR> <ITERATION RANGE> <OPT BY>
                   <OPT WHILE>

<OPT FOR> ::=
            | FOR <ID>

<ITERATION RANGE> ::= <OPT FROM> <OPT TO>
                    | OVER <TYPE>

<OPT FROM> ::=
             | FROM <EXPRSN>

<OPT TO> ::=
           | UP TO <EXPRSN>
           | DOWN TO <EXPRSN>

<OPT BY> ::=
           | BY <EXPRSN>

<OPT WHILE> ::=
              | WHILE <EXPRSN>

<OPT STMT SEQ> ::=
                 | <OPT STMT SEQ> <STMT> ;
```

EXAMPLES: (statements within DO ENDDO are omitted for brevity.)

1.
```
FOR i FROM 1 UP TO 10 BY 2
   DO
     .
     .
     .
   ENDDO;
```

2.
```
FOR j FROM 5 DOWN TO 0 BY 1
   DO
     .
     .
     .
   ENDDO;
```

3.

```
FOR j FROM 5    % equivalent to previous example
   DO           % due to default assumptions
     .
     .
     .
   ENDDO;
```

4.

```
FOR j FROM -10 UP TO 10 WHILE a < b
   DO
     .
     .
     .
   ENDDO;
```

5.

```
WHILE (x <= y) | (z = 0)    % loops while expression
   DO                       % is true.
     .
     .
     .
   ENDDO;
```

6.

```
DO          % Equivalent to DO WHILE TRUE
  .         % i.e., infinite loop
  .
  .
ENDDO;
```

7.

```
FOR i OVER index_type
   DO
     .
     .
     .
   ENDDO;
```

8.

```
OVER symbolic_range_type
   DO
     .
     .
     .
   ENDDO;
```

DESCRIPTION

1. The iterative statement is used to provide controlled iteration of one or more statements. Options are provided so that loop termination may be controlled by an index variable, a boolean expression, an EXIT statement or a combination of these.

2. The FOR <ID> phrase is used to declare a variable to be used as an index within the loop. The type of this variable will be the same as the type of the <EXPRSN> following FROM, UP TO, or DOWN TO. If OVER <TYPE> is used, the type of the index variable will be <TYPE>. The index variable is local to the loop and may not be accessed outside the loop (i.e., outside of DO ... ENDDO). If the same index variable is used with a FOR phrase on an inner loop, the inner variable supercedes the outer throughout the scope of the inner loop. These scope rules are the same as for blocks and are described in "Scope Rules" on page 51.

3. Iteration over a range can be controlled either by specifying the start and end points with FROM and UP TO (DOWN TO) or by designating a type over which the iteration is to range. The <EXPRSN> that follows FROM, UP TO, or DOWN TO must yield a value of type numeric or symbolic range. The <TYPE> following OVER must also be either a numeric or symbolic range. If both FROM and UP TO (DOWN TO) are given, the type of <EXPRSN> must be the same for both.

4. The <EXPRSN> following WHILE must yield a boolean value. The <EXPRSN> following BY must yield a numeric value.

5. Different forms of iteration control may be obtained by different combinations of the optional phrases indicated in the syntax description. Whenever phrases are omitted, the following defaults are assumed:

Phrase Omitted	Default
FOR <ID>	internal index created
FROM <EXPRSN>	**FROM** 0
<OPT TO>	**DOWN TO** 0
BY <EXPRSN>	**BY** 1. If <ITERATION RANGE> is omitted, **BY** 0 is assumed.
WHILE <EXPRSN>	**WHILE TRUE**

6. If OVER <TYPE> is given it is assumed to be equivalent to:

```
FROM low_limit UP TO high_limit
```

   where low_limit and high_limit are respectively the lower and upper extremes of the range when <TYPE> is a numeric range. When <TYPE> is a symbolic range, the loop is executed once for each element of the range. For example, given

```
TYPE  n  {3 TO 9};
```

   then "OVER n" is equivalent to "FROM 3 UP TO 9".

7. Omission of the <LOOP CONTROL> phrase, as in example 6 above, causes an infinite loop to be executed.

8. The iteration loop terminates when either the <ITERATION RANGE> has been exhausted or the WHILE <EXPRSN> becomes false. If both phrases are present, the index variable is checked before the WHILE <EXPRSN> is evaluated.
9.

10. The order in which the clauses of a FOR loop are computed depends upon the implementation.

    In the DMS implementation, the execution of a FOR loop is as follows:

    a. evaluate FROM <EXPRSN>

    b. evaluate UP TO <EXPRESN>

    c. IF current loop index > UP TO <EXPRSN> THEN
       exit loop

    d. evaluate WHILE <EXPRSN>

    e. IF WHILE <EXPRESN> is false THEN
       exit loop

    f. execute loop body

    g. evaluate BY <EXPRSN>

    h. add BY <EXPRSN> to loop index

    i. goto step b.

    In the IBM/370 implementation, the execution of a FOR loop is as follows:

    a. evaluate FROM <EXPRSN>

    b. evaluate UP TO <EXPRSN>

    c. evaluate BY <EXPRSN>

    d. IF current loop index > UP TO <EXPRSN> THEN exit loop

    e. evaluate WHILE <EXPRSN>

    f. IF WHILE <EXPRESN> is false THEN exit loop

    g. execute loop body

    h. add BY <EXPRSN> to loop index

    i. goto step d.

EXAMPLES

The following examples illustrate some typical forms of iterative statements:

1.

```
e: DO     % forever
       i + 10 -> i;
```

```
        IF i < 5 THEN
           EXIT e;
        ENDIF;
     ENDDO e;
```

2.

```
DCL  d  DESC  OF  integer;
  :
  :
  :
FOR  i  UP TO  UPB d
   DO
      0 -> d[i];
   ENDDO;
```

3.

```
TYPE day {sun,mon,tues,wed,thurs,fri,sat};

DCL calls_per_day TABLE[day] OF integer;
DCL calls_per_week, max integer INIT 0;

FOR any_day OVER day
   DO
    calls_per_week + calls_per_day[any_day]
                                -> calls_per_week;
   ENDDO;

FOR busy_day FROM tues UP TO thurs
   DO
      IF calls_per_day[busy_day] > max THEN
         calls_per_day[busy_day] -> max;
      ENDIF;
   ENDDO;
```

4.

```
% linear table search

FOR i OVER port_index WHILE port_table[i] ^= port
   DO
      i -> port_table_index;
               % to have i available outside the loop
               % it must be assigned to a variable
               % (port_table_index) declared outside
               % the loop.
   ENDDO;
```

10.0 BIND and WITH Declarations

The BIND and WITH declarations appear in a separate section because they combine some of the properties of declarations and of executable statements. They are used to allow abbreviation of lengthy names and to permit significant optimization to be made by the compiler.

SYNTAX

```
<DCL STMT> ::= <TYPE DCL STMT>
             | <DATA DCL STMT>
             | <WITH DCL STMT>
             | <BIND DCL STMT>

<WITH DCL STMT> ::= WITH <STORAGE REF LIST>

<STORAGE REF LIST> ::= <STORAGE REF>
                     | <STORAGE REF LIST>, <STORAGE REF>

<BIND DCL STMT> ::= BIND <BIND LIST>

<BIND LIST> ::= <BIND>
              | <BIND LIST> ,  <BIND>

<BIND> ::= <ID> TO <STORAGE REF>
         | <ID> TO <STORAGE REF> AS <TYPE>
```

EXAMPLES

1.
```
BIND x TO structure_variable; % when selecting elements
                                % of structure_variable
                                % x can be used instead
```

2.
```
WITH structure_variable;      % can now reference elements
                                % within structure_variable
                                % without qualification of
                                % structure_variable.
```

3.
```
WITH table_of_structs[i],     % note that the first two
          ptr_to_struct@;       % items after WITH involve
          struct2.inner_struct, % run-time access to variables
                                % table_of_structs[i], and
                                % prt_to_struct.
```

DESCRIPTION

1. The WITH construct of PROTEL should never be used. It has been deemed inherently bad, and is being removed from the language. It has already been removed from some of the PROTEL compilers. The primary reason it is documented here is because there still exist some WITH statements in old source code.

2. The WITH declaration is used to abbreviate references to an element of a structure. As an example, the structure element declarations on the left can be replaced by the WITH declaration and abbreviated reference on the right:

```
                  WITH s1;
s1.e              e
s1.s2.e           s2.e              % s2 is substructure in
                                    % s1.

                  WITH p@.s2;
p@.s2.e           e                 % p is a  pointer to a
                                    % structure containing
                                    % the substructure s2.

                  WITH t1[i+1]@;    % t1 is a table of
t1[i+1]@.e        e                 % pointers to a
t1[i+1]@.t2[j]    t2[j]             % structure with
                                    % elements e and table t2
```

3. WITH and BIND declarations may appear anywhere local declarations are allowed.

4. A WITH declaration is illegal if it allows use of an identifier which is the same as any other identifier declared in the same block. The use of such a WITH declaration constitutes a duplicate declaration of the ambiguous element. For example, the following WITH declaration would be illegal:

```
BLOCK
  DCL i integer;
  DCL s
    STRUCT
       i  integer,
       b  BOOL
    ENDSTRUCT;
  WITH s;              % this is illegal since a ref-
                       % erence to i would be ambiguous.
```

   Note that if i had been a global variable the WITH declaration would be allowed as it would constitute a local declaration rather than a multiple declaration of i.

5. The BIND declaration is similar to, but more general, than the WITH declaration: BIND may be used with any addressable storage reference whereas WITH can be used only with a structure or elements thereof. A BIND statement gives a name to an expression (the bound expression), whereas a WITH statement does not. Since the bound expression is referenced by name, the bound expression can be easily located by examining a cross-reference. No such facility exists for WITH.

   BIND is safer than WITH, since changes to a WITH'ed structure can cause identifiers within the scope of a WITH to become references to different data objects, with no warnings to the programmer. When bound by a BIND statement, the same references will fail to compile, drawing the attention of the programmer to statements which are affected by the structure changes.

6. BIND associates an identifier with a storage reference: whenever the identifier is encountered, it represents the storage reference.

7. When binding to an area, the AS clause of the BIND is used to specify the desired sub-area type. For example

```
DCL area1 father; % these types are declared in example 1
                  % in "Areas".

BIND x TO area1 AS son1;
```

   Area refinement is the only valid use of the AS clause of a BIND statement.

8. If the <STORAGE REF> used in a WITH or BIND declaration involves indexing or pointer dereferencing, then a local copy of the reference will be made for efficiency. When a WITH or BIND is executed, a local copy of the computed address of the WITHed or bound ("BINDed") storage reference is made. Consequently, no changes to the WITH or BIND expression will change the target of references of the WITH or the bound identifier.

EXAMPLES

a.

```
                    WITH t[i];      BIND j TO t[i];
t[i].e -> x;        e -> x;         j.e -> x;
i+1 -> i;           i+1 -> i;       i+1 -> i;
t[i].e -> y;        e -> y;         j.e -> y;
```

   At the end of the left most sequence, x and y could have different values. In the other two sequences, they would have the same value.

b.

```
0 -> t[0].e -> t[1].e;      |  0 -> t[0].e -> t[1].e;
0 -> i;                     |  0 -> i;
BLOCK                       |  BLOCK
                            |     BIND j TO t[i];
   1 -> t[i].e;             |     1 -> j.e;
   i + 1 -> i;              |     i + 1 -> i;
   0 -> t[i].e;             |     0 -> j.e;
ENDBLOCK;                   |  ENDBLOCK;
      |
```

   At the end of the left sequence, t[0].e = 1 and t[1].e = 0, while at the end of the right sequence t[0].e = 0 and t[1].e = 0.

11.0 TYPE COMPATIBILITY

This section outlines the rules governing the allowed operations between data elements.

11.1 OPERATOR/OPERAND PAIRS

For the purpose of describing the type compatibility rules, the binary operators (operators accepting two operands) can be divided into six classes:

1. Arithmetic +, -, /, *, MOD, <<, >>
2. Boolean &, |, !
3. Equality =, ^=
4. Relational >, <, >=, <=
5. Set INCL, NOTINCL, &, |, !
6. Assignment ->

The operand types valid for each class of operator are given in Figure 3.

NOTE:

1. The type of a dereferenced pointer is the type of the item pointed to. E.g.,

```
DCL p PTR TO x;
```

   the type of p@ is x.

2. The type of a structure or area element is the type of the element itself. e.g.:

```
DCL s
   STRUCT
      e  x,
      f  int
   ENDSTRUCT;
```

   the type of s.f is int.

3. The type of a table or descriptor element is the type of the element itself; e.g.,

Operand Type	Operator Class Allowed
Numeric range	arithmetic, equality, relational, assignment, boolean
Symbolic range	equality, assignment
Pointer	equality, assignment
Boolean	boolean, equality, assignment
Sets	set, equality, assignment
Structures	equality, assignment
Tables	equality, assignment
Descriptors	equality, assignment
Areas	equality, assignment.

Figure 3. Operands and Operators

```
DCL t[0 TO n] OF x;
```

the type of t[i] is x.

11.2 Primitive, Root, and Defined Types

All user-defined types are built from PROTEL primitive types. The primitive types are:

1. numeric range
2. symbolic range
3. **BOOL**
4. **SET**
5. **TABLE**
6. **DESC**
7. **STRUCT**
8. **AREA**
9. **PTR**
10. **PROC**

A root type is a primitive type along with whatever specifications are necessary to define the type. Consider T1 in Figure 4. The root type of T1 is "numeric range 0 TO 255". The primitive type of T1 is "numeric range".

A type is a defined type if its type is given by a type identifier. T1 is not a defined type, as it is defined in terms of primitive types. An example of a defined type is T2 in Figure 4. T2 is not defined in terms of primitive types; it is defined in terms of another type. We say that T1 is the definition type of T2.

We can see that T1 and T2 have the same root type, "numeric range {0 TO 255}". T1 and T2 have the same primitive type as well, "numeric range".

A defined type always has a type name, so the term "defined type" is synonymous with the term "named type". Note that in a type declaration and a variable declaration there is a field for a type specification. The type specification of T1 is "{0&rblTO&rbl255}". The type specification for T2 is "T1".

In a DCL statement, the field for the type is identically a type specification field. Consider the declarations of X1 and X2 in Figure 4. The type of X1 is a defined type. Its type is defined in terms of T1.

We can now define root type recursively: The root type of a defined type is the root type of its definition type. The root type of a non-"defined type" is its type specification.

In the cross reference of a LISTING file generated by the current compiler, there are two fields for each entry which refer to defined and primitive types. Unfortunately, they are not labelled correctly. The first entry is labelled TYPE. For a defined type, this entry contains the name of the definition type. For a type which is not a defined type, it lists the primitive type.

The second entry is a field labelled ROOT. For a defined type, it contains the primitive type of the identifier. For a type which is not a defined type, it is empty.

Note that the cross reference does not give root types, as defined above. For example, there is no way from the cross reference to derive that B1 is a table of T1, nor can we find its subscript type or range in the cross reference. Similarly, for a STRUCT, the cross reference does not indicate the names of the fields of the STRUCT.

```
    1 |SECTION Types;
    2 |
    3 |TYPE
    4 |   T1 {0 TO 255},
    5 |   T2 T1,
    6 |   T3 T2;
    7 |
    8 |DCL
    9 |   X1 T1,
   10 |   X2 {0 TO 255};
   11 |
   12 |TYPE
   13 |   B1 TABLE[0 TO 10] OF T1,
   14 |   B2 B1,
   15 |   B3 TABLE[0 TO 10] OF T1;

C R O S S  - -  R E F E R E N C E

IDENTIFIER DEFINITION SEG-TYPE OFFSET SIZE   CLASS TYPE    ROOT

B1            13              0000.0 0005.8 TYPE  TABLE
B2            14              0000.0 0005.8 TYPE  B1      TABLE
B3            15              0000.0 0005.8 TYPE  TABLE
T1             4              0000.0 0000.8 TYPE  NUMERIC
T2             5              0000.0 0000.8 TYPE  T1      NUMERIC
T3             6              0000.0 0000.8 TYPE  T2      NUMERIC
X1             9      SHARED  0000.0 0001.0 VAR   T1      NUMERIC
X2            10      SHARED  0001.0 0001.0 VAR   NUMERIC
```

Figure 4. Primitive, root and defined types

11.3 Type Compatibility Rules

A further constraint on the use of the binary operators of "Type Compatibility" is that the operands be type-compatible in accordance with the type-compatibility rules.

1. NUMERIC

   Any two numeric range types are compatible.[19]

2. BOOLEAN

   Any two types having primitive type BOOL are compatible.

3. DEFINED TYPES

   Two types having the same type name are compatible.

```
TYPE px PTR TO s;
DCL p px;
DCL q px;
```

   p and q are type-compatible since they have the same defined type name px.

4. ANONYMOUS TYPES

   An anonymous type is a type with no name. An anonymous type occurs whenever a variable or constant is not declared as having a type declared in a TYPE statement (i.e., not declared in terms of a defined type). Two anonymous types are compatible if they are defined in the same identifier list.

```
DCL u, v PTR TO s;
```

   u and v both have anonymous types, since the type has no name. They are type-compatible since they are declared in the same identifier list.

   This rule for compatibility does not prevent an anonymous type from being compatible with some other type by any other rule.

---

[19] Recall that a single character is a subset of numeric range.

5. POINTERS

Two pointer types are compatible if the types to which they point are compatible.

```
DCL c PTR TO s; DCL d PTR TO s;
```

c and d are type compatible since they are both pointers to s.

6. TABLES

Two table types are compatible if they satisfy:

a. they have the same number of elements (i.e., their TDSIZEs are equal);

b. their elements have the same width (i.e., their strides are equal);

c. their element types are compatible. Such as

```
DCL e TABLE[1 TO 5] OF {0 TO 8};
DCL f TABLE[5 TO 9] OF {0 TO 9};
```

where e and f are type-compatible since they:

a. both have TDSIZE = 5; and

b. both have stride = 4; and

c. both have elements of primitive type "numeric range."

7. DESCRIPTORS

There are two sets of rules for descriptor compatibility corresponding to the two parts of a descriptor - the descriptor part and the table part. Given

```
TYPE chardesc DESC OF char;
DCL d chardesc;
```

the type of a dereference of d is some table type. For example, the type of d[] is "TABLE [0 TO UPB d] OF char" and the compatibility rules for tables apply.

For the descriptor part, d itself, we say that two descriptors are compatible if they have the same stride, compatible index types, and compatible elements.

8. SELF-DEFINING CONSTANTS

a. A self-defining constant (i.e., a constant which is not defined in a declaration) is compatible with a type t if the root type of t is the same as the type of the constant.

The types of constants are given in Figure 5.

A tuple constant is compatible with a <TYPE> if the corresponding elements in the <TYPE> and the tuple are compatible in type and number.

b. Two simple self-defining constants are compatible if they have the same type. Two aggregate self-defining constants (i.e., tuples) are compatible if their corresponding elements are compatible in type and number.

9. PROCEDURES

Procedure types are compatible only when they satisfy:

a. both are functions or both are procedures (i.e., either both or neither have a RETURNS clause in their declaration).

b. in the case of functions, the types of value returned by each must be compatible.

c. their formal parameter lists are of equal length.

d. corresponding parameters in the parameter lists have the same name and passing mode, and are of compatible types.

e. A standard, QUICK, or NONTRANSPARENT procedure can be compatible only with another procedure of the same class. Procedure classes QUICK and NONTRANSPARENT are specific to the DMS/NT40 implementation, and are discussed in "Procedure Classes".

10. PROCEDURE PARAMETERS

In the case of a procedure call the compatibility of actual and formal parameters is governed by the rules for the assignment operator, where the actual parameter is considered to be the left hand side and the formal parameter the right hand side.

11. SETS

Two set types are compatible if their element types are compatible.

12. STRUCTURES

Two struct types are compatible when:

a. they have the same number of fields;

b. corresponding fields have the same name;

c. corresponding fields are compatible.

13. AREAS

Two area types are compatible when:

a. they have the same bit width;

b. they have the same number of fields;

c. corresponding fields have the same name;

d. corresponding fields are compatible.

Area types are also assignment-compatible when the source is a refinement of the destination.

Constant	Type	Example
hex or decimal number	numeric range	#2F, 123, 'A'
string	table, set, or structure of numeric elements	'ABC'. (equivalent to {'A','B','C'}).
tuple	table or structure type with element types the same as the tuple elements. SET type which contains the tuple elements	{1,2,3} or {1, **NIL**, **TRUE**}. **TYPE** b **SET** {c, d}; the type of {c} is b.
symbolic range element	symbolic range type which contains the element	**TYPE** a {x, y}; the type of x and y is a.
TRUE, **FALSE**	boolean	
NIL	pointer, descriptor or procedure	

Figure 5. Types of Constants

12.0 INTRINSIC PROCEDURES

SYNTAX

```
<DCL GROUP> ::= <CONSTANT DCL GROUP><CONSTANT INIT>

<CONSTANT DCL GROUP> ::= PROC ( <PARM LIST> ) <OPT RETURNS>

<CONSTANT INIT> ::= IS INTRINSIC

<PARM> ::= <OPT PASSING MODE><ID LIST><TYPE>
         | <LITERAL PARM>

<LITERAL  PARM> ::= LITERAL ( <CONSTANT EXRSN> ,
                    <CONSTANT EXPRSN> )

<OPT PASSING MODE> ::=
                     | REF
                     | UPDATES
                     | OPERAND ( <CONSTANT EXPRSN> )
                     | VARIABLE ( <CONSTANT EXPRSN> )

<OPT RETURNS> ::=
                | RETURNS <TYPE>
```

EXAMPLES

```
DCL read_scanner PROC(LITERAL(8,102),
                      OPERAND(8)   priority int,
                      VARIABLE(24) line_size int,
                      line_no      int)
                 RETURNS BOOL IS INTRINSIC;
```

DESCRIPTION

1. Intrinsic procedures are used to generate specific machine instructions to perform specialized functions not supported directly in PROTEL.

2. The LITERAL(len, val) parameter defines a literal len bits in length and having a value val. This can be used to create an instruction op-code by using a LITERAL parameter whose value is a machine op-code. When an intrinsic procedure is invoked there is no corresponding actual parameter for a LITERAL parameter.

3. The OPERAND(len) parameter defines an immediate field in the instruction of length len bits. The value in this field is provided by a corresponding actual parameter which must be a simple compile time constant.

4. The VARIABLE(len) parameter defines an immediate field in the instruction of length len bits. The corresponding actual parameter must be a variable, whose base-offset address provides the value for this immediate field. Accepted values for len are implementation-dependent.[20]

5. The value, REF, and UPDATES parameters are handled as they are for regular procedures.

For example, the intrinsic procedure, read_scanner, defined in example 1 above could be invoked with:

```
read_scanner(3,size,no) -> status;
   % note only 3 actual parameters
```

[20] PROTEL/68K has the following restrictions on INTRINSIC procedures:

a. VARIABLE(len) parameter passing is not supported;

b. LITERAL and OPERAND parameters must precede any other parameters;

c. INTRINSIC procedures must be of an even number of bytes.

13.0 TYPEDESC, VARDESC, and REFDESC

The TYPEDESC and VARDESC features of the language provide a type and variable description facility.

The operator TYPEDESC, when applied to a type, produces a description of that type. This description is in the form of a PROTEL structure of type $TYPEDESC. This type is automatically imbedded by the compiler and need not be declared by the user. Its definition is implementation-dependent. For example:

```
DCL ty $TYPEDESC;
TYPE char {0 TO 255};
TYPEDESC char -> ty;
```

The VARDESC facility is implemented through a special type $VARDESC. This type is only valid as a formal parameter type to a procedure. The corresponding actual parameter must be a storage reference. On procedure invocation, the address of the storage reference and some information about its type are entered in the $VARDESC parameter. The address is of type $UNIVERSAL_PTR.

$VARDESC is automatically imbedded by the compiler and need not be declared by the user. Its definition is implementation-dependent.

The compatibility rules for $UNIVERSAL_PTR are: A pointer to any PROTEL type may be assigned to a variable of type $UNIVERSAL_PTR but not vice versa. All other operations require that $UNIVERSAL_PTR be appropriately cast.

A REFDESC is identical to a VARDESC, but is intended for read-only access to the variable to which it gives access. REFDESC is supported only in the DMS implementation.

In the DMS implementation, the definitions of $TYPEDESC, $VARDESC and $REFDESC are in module PROTDEFS. In the IBM/370 implementation, the module name is DEFNS370.

14.0 PROTEL 2026 Development Tools

14.1 PROTEL 2026 Compiler

Invoke the compiler from a MacOS or UNIX / Linux command line with:

```
> protel [-o outfile] sourcefile(s) [—classical]
```

`-o outfile`: Specify output (executable or object file). Defaults to a.out.
sourcefile(s): One or more .pt files (PROTEL source).

`--classical`: Enables case-insensitivity for identifiers (compatibility with original Nortel DMS). Keywords remain UPPERCASE.

The compiler uses GCC backend: Generates C intermediates, compiles to native code. Supports -g (debug), -c (compile only), etc., passed to GCC. PROTEL PROCs are C-callable; use extern in C for vice-versa.

14.2 The PROTEL Beautifier - Pb

Pb is a lexical beautifier (or “pretty-printer”) for PROTEL source code. It reads PROTEL text, performs a simple lexical analysis, and rewrites every reserved keyword in UPPERCASE. With the --bold option it additionally surrounds each keyword with ANSI SGR escape sequences (\033[1m … \033[0m) so that the keywords appear in bold on terminals and in editors that support ANSI escape sequences.

Pb knows the complete set of PROTEL reserved words given in Appendix A of this manual, together with the compiler directives ($EJECT, $LI, $OBJECT, …). A keyword is only uppercased (and optionally bolded) when it appears as a token outside a comment or string literal. Comments (introduced by % and running to the end of the line) and string literals (delimited by apostrophes, with '' representing an embedded apostrophe) are left completely untouched. All other tokens — user identifiers, numeric literals, operators (including ->), punctuation, and whitespace — are preserved exactly as they were written.

The program is intentionally a pure filter. It can be used with redirection, pipes, or explicit file-name arguments for both input and output. If the input contains very few PROTEL keywords, Pb emits a warning on standard error but still produces the (minimally modified) output.

The original Pb was created in the early 1990s by Vincente D’Ingianni using C and Lex; however, the original source code was lost. This new version for PROTEL 2026 is written in Python for ease of portability.

Invocation

Pb may be invoked in any of the following ways:

```
> Pb < infile > outfile
> Pb infile outfile
> Pb --bold < infile | less
```

```
> cat infile | Pb
> ./Pb myprog.protel > myprog.pretty.protel
```

After chmod +x Pb the program can be executed directly because it begins with the standard Python 3 shebang (#!/usr/bin/env python3).

Arguments and Options

```
usage: Pb [-h] [-b] [--version] [input] [output]
```

- `input`
  Input PROTEL file (default: read from standard input). Use “-” to read explicitly from stdin.

- `output`
  Output file (default: write to standard output). Use “-” to write explicitly to stdout.

- `-b, --bold`
  Wrap each uppercased keyword in ANSI bold escape sequences (\033[1m … \033[0m).

- `--version`
  Print the program identification string and exit.

- `-h, --help`
  Print a short usage message and exit.

Version string

```
Pb 1.0 (PROTEL Beautifier)
```

The keyword list used by Pb is taken directly from this PROTEL Reference Manual (Appendix A),.

14.2 EMACS PROTEL Mode

Originally created by the late Bill Conn, a new EMACS PROTEL Mode is planned in his honor.

PROTEL Mode provides capitalization, text highlighting, and proper indentation for EMACS users.

Appendix A: Keywords and Symbols

AREA	**NONTRANSPARENT**
AS	**NOTINCL**
BIND	**OF**
BLOCK	**OPERAND**
BOOL	**OUT**
BY	**OVER**
CASE	**OVLY**
CAST	**PACK**
CLASS	**PERPROCESS**
DCL	**PRIVATE**
DEFINITIONS	**PROC**
DESC	**PROTECTED**
DO	**PTR**
DOWN	**QUICK**
ELSE	**REF**
ENDAREA	**REFDESC**
ENDBLOCK	**REFINES**
ENDCASE	**RETURN**
ENDCLASS	**RETURNS**
ENDDO	**SECTION**
ENDIF	**SELECT**
ENDOVLY	**SET**
ENDSELECT	**SELF**
ENDSTRUCT	**SHARED**
ENTRY	**STRUCT**
EXIT	**TABLE**
FALSE	**TDSIZE**
FAST	**THEN**
FOR	**TO**
FORWARD	**TRUE**
FROM	**TYPE**
IF	**TYPEDESC**
IN	**UNRESTRICTED**
INCL	**UP**
INIT	**UPB**
INTERFACE	**UPDATES**
INTRINSIC	**USES**
IS	**VAL**
LITERAL	**VARDESC**
METHOD	**VARIABLE**
MOD	**WHILE**
NIL	**WITH**

Figure 6. Reserved Words: These keywords are reserved and may not be used as identifiers.

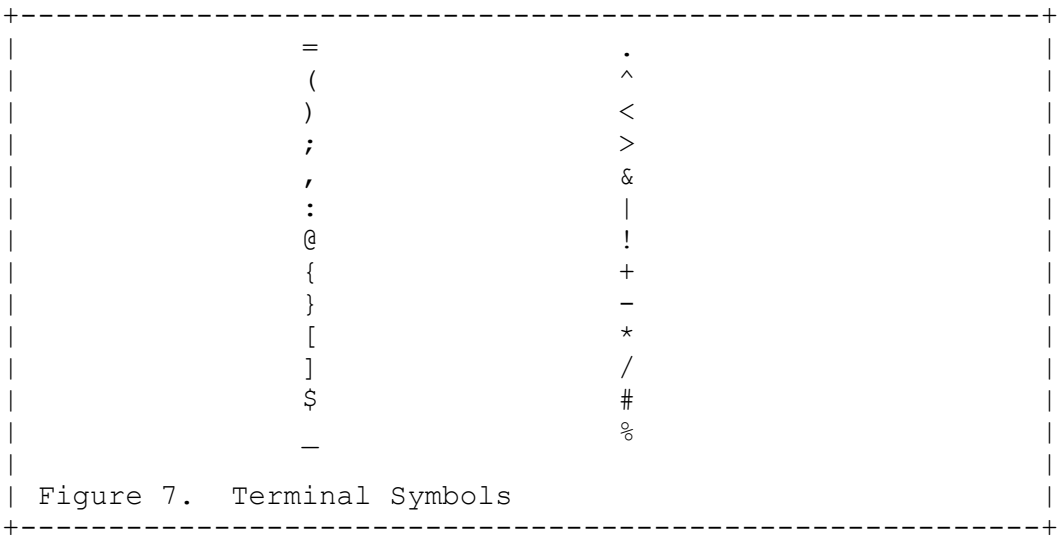


Figure 7. Terminal Symbols

Appendix B: Language Grammar

```
<ACTUAL PARM LIST> ::= <EXPRSN>
                     | <ACTUAL PARM LIST> , <EXPRSN>

<AGGREGATE TYPE> ::= <STRUCT TYPE>
                   | <SET TYPE>
                   | <TABLE TYPE>
                   | <DESC TYPE>
                   | <AREA TYPE>

<AREA BODY> ::=
              | <DCL LIST>

<AREA HEADER> ::= <AREA OPTIONS> AREA ( <EXPRSN> )
                | <AREA OPTIONS> AREA REFINES <DEFINED TYPE>

<AREA OPTIONS> ::=
                 | UNRESTRICTED

<AREA TYPE> ::= <AREA HEADER> <AREA BODY> ENDAREA

<BIND> ::= <ID> TO <STORAGE REF>
         | <ID> TO <STORAGE REF> AS <TYPE>

<BIND DCL STMT> ::= BIND <BIND LIST>

<BIND LIST> ::= <BIND>
              | <BIND LIST> , <BIND>

<BLOCK> ::= BLOCK <OPT STMT SEQ> ENDBLOCK <OPT END LABEL>

<BOOL CONSTANT> ::= TRUE
                  | FALSE

<BOOL TYPE> ::= BOOL

<BOUNDS> ::= [ <CONSTANT SUBRANGE> ]
           | [ <DEFINED TYPE> ]

<CASE BODY> ::= <IN RANGE CASES> <OPT OUT CASE>

<CASE HEAD> ::= CASE <EXPRSN> IN

<CASE LABEL> ::= <TUPLE> :

<CASE STMT> ::= <CASE HEAD> <CASE BODY> ENDCASE <OPT END LABEL>

<COMPILATION UNIT> ::= <SECTION>

<CONSTANT DCL GROUP> ::= <ID LIST> <TYPE>
```

```
<CONSTANT> ::= <NUMERIC CONSTANT>
             | <BOOL CONSTANT>
             | <TUPLE>
             | <PTR CONSTANT>

<CONSTANT EXPRSN> ::= <EXPRSN>

<CONSTANT INIT> ::= IS <UNLABELLED STMT>
                  | IS FORWARD
                  | IS ENTRY
                  | IS INTRINSIC

<CONSTANT SUBRANGE> ::= <CONSTANT EXPRSN> TO <CONSTANT EXPRSN>

<DATA DCL STMT> ::= PROTECTED <DCL LIST>
                  | SHARED <DCL LIST>
                  | PRIVATE <DCL LIST>
                  | DCL <DCL LIST>

<DCL> ::= <DCL GROUP> <OPT DATA INIT>
        | <CONSTANT DCL GROUP> <CONSTANT INIT>
        | <OVLY GROUP>

<DCL GROUP> ::= <ID LIST> <TYPE>

<DCL LIST> ::= <DCL>
             | <DCL LIST> , <DCL>

<DCL STMT> ::= <TYPE DCL STMT>
             | <DATA DCL STMT>
             | <WITH DCL STMT>
             | <BIND DCL STMT>

<DEFINED TYPE> ::= <QUALIFIED ID>

<DEREFERENCE> ::= <STORAGE REF> @

<DESC TYPE> ::= DESC <OPT BOUNDS> OF <TYPE>

<EXIT STMT> ::= EXIT <ID>

<EXPRSN> ::= <SECONDARY>
           | <EXPRSN> <OPERATOR> <SECONDARY>

<ID LIST> ::= <ID>
            | <ID LIST> , <ID>

<IF STMT> ::= IF <EXPRESN> <THEN PART> <OPT ELSE PART> ENDIF
              <OPT END LABEL>

<IN RANGE CASES> ::= <CASE LABEL>
                   | <IN RANGE CASES> <CASE LABEL>
```

```
                    | <IN RANGE CASES> <STMT> ;


<INDEXING> ::= <STORAGE REF> [ <EXPRSN> ]

<ITERATION RANGE> ::= <OPT FROM> <OPT TO>
                      | OVER <TYPE>

<ITERATIVE STMT> ::= <LOOP CTRL> DO <OPT STMT SEQ> ENDDO
                       <OPT END LABEL>

<LABEL> ::= <ID> :

<LITERAL PARM> ::= LITERAL ( <CONSTANT EXPRSN> ,
                     <CONSTANT EXPRSN> )

<LOOP CTRL> ::= <OPT FOR> <ITERATION RANGE> <OPT BY> <OPT WHILE>

<MODULE TYPE> ::=
                  |  FAST
                  |  PERPROCESS
                  |  DEFINITIONS

<MULTIPLE> ::= <STORAGE REF> [ <OPT SUBRANGE> ]

<NUMERIC CONSTANT> ::= <DECIMAL NUMBER>
                       | <HEX NUMBER>

<NUMERIC RANGE> ::= { <CONSTANT SUBRANGE> }

<OPT ACTUAL PARM LIST> ::=
                           | <ACTUAL PARM LIST>

<OPT BOUNDS> ::=
                | <BOUNDS>

<OPT BY> ::=
           | BY <EXPRSN>

<OPT DATA INIT> ::=
                    | INIT <UNLABELLED STMT>

<OPT DCL STMT SEQ> ::=
                     | <OPT DCL STMT SEQ> <DCL STMT> ;

<OPT ELSE PART> ::=
                    | ELSE <OPT STMT SEQ>

<OPT END LABEL> ::=
                    | <ID>

<OPT EXEC STMT SEQ> ::=
                      | <OPT EXEC STMT SEQ> <STMT> ;
```

```
<OPT FOR> ::=
            | FOR <ID>


<OPT FROM> ::=
             | FROM <EXPRSN>

<OPT OUT CASE> ::=
                 | OUT <OPT STMT SEQ>

<OPT PACKAGE> ::=
                | IN <ID>

<OPT PACKING> ::=
                | PACK ( <CONSTANT EXPRSN> )

<OPT STMT SEQ> ::= <OPT DCL STMT SEQ> <OPT EXEC STMT SEQ>

<OPT SUBRANGE> ::=
                 | <EXPRSN> TO <EXPRSN>

<OPT TO> ::=
           | UP TO <EXPRSN>
           | DOWN TO <EXPRSN>

<OPT USES_IMPL LINK> ::=
                       |  USES <ID LIST>
                       |  OF <ID LIST>

<OPT WHILE> ::=
              | WHILE <EXPRSN>

<OVLY BODY> ::= <CASE LABEL> <OVLY DCL LIST>
              | <OVLY BODY> <CASE LABEL> <OVLY DCL LIST>

<OVLY DCL> ::= <DCL GROUP>
             | <OVLY GROUP>

<OVLY GROUP> ::= <OVLY HEAD> <OVLY BODY> ENDOVLY

<OVLY HEAD> ::= OVLY <TYPE>

<OVLY DCL LIST> ::=  <OVLY DCL>
      |                <OVLY DCL LIST> , <OVLY DCL>

<OPT PCLASS> ::=
               | QUICK
               | NONTRANSPARENT

<OPT PARMS> ::=
              | <PARM LIST>

<OPT PASSING MODE> ::=
                     | REF
```

```
                    | UPDATES
                    | OPERAND ( <CONSTANT EXPRSN> )
                    | VARIABLE ( <CONSTANT EXPRSN> )


<OPT REPETITION FACTOR> ::=
                          | <PRIMARY>

<OPT RETURNS> ::=
                | RETURNS <TYPE>

<OPT TUPLE BODY LIST> ::=
                        | <TUPLE BODY LIST>

<OPT VAL> ::=
            | VAL

<PARM> ::= <OPT PASSING MODE> <DCL GROUP>
         | <LITERAL PARM>

<PARM LIST> ::= <PARM>
              | <PARM LIST> , <PARM>

<PRIMARY> ::= <CONSTANT>
            | <TUPLE>
            | <STORAGE REF>
            | PTR <STORAGE REF>
            | DESC <STORAGE REF>
            | ( <EXPRSN> )

<PROC CALL> ::= <STORAGE REF> ( <OPT ACTUAL PARM LIST> )

<PROC TYPE> ::= <PROC> ( <OPT PARMS> ) <OPT RETURNS>

<PROC> ::=  <OPT PCLASS> PROC

<PTR CONSTANT> ::= NIL

<PTR TYPE> ::= PTR TO <TYPE>

<RANGE TYPE> ::= <SYMBOLIC RANGE>
               | <NUMERIC RANGE>

<RETURN STMT> ::= RETURN
                | RETURN <EXPRSN>

<SECONDARY> ::= <PRIMARY>
              | <OPERATOR> <SECONDARY>

<SECTION> ::= <SECTION HEAD> <OPT DCL STMT SEQ>

<SECTION HEAD> ::= <MODULE TYPE> <SECTION TYPE> <ID>
                   <OPT PACKAGE> <OPT USES_IMPL LINK> ;
```

```
<SECTION TYPE> ::= INTERFACE
                 |  SECTION
<SET TYPE> ::= SET <SIMPLE TYPE>

<SELECT BODY> ::= <CASE BODY>

<SELECT HEAD> ::= SELECT <EXPRSN> IN

<SELECT STMT> ::= <SELECT HEAD> <SELECT BODY> ENDSELECT
                  <OPT END LABEL>

<SELECTION> ::= <STORAGE REF> . <ID>

<SIMPLE TYPE> ::= <RANGE TYPE>
                | <BOOL TYPE>
                | <PTR TYPE>
                | <DEFINED TYPE>

<STMT> ::= <UNLABELLED STMT>
         | <LABEL> <UNLABELLED STMT>

<STORAGE REF> ::= <QUALIFIED ID>
                | <SELECTION>
                | <DEREFERENCE>
                | <INDEXING>
                | <MULTIPLE>
                | <PROC CALL>
                | CAST <EXPRESN> AS <TYPE>

<STORAGE REF LIST> ::= <STORAGE REF>
                     | <STORAGE REF LIST> , <STORAGE REF>

<STRUCT TYPE> ::= STRUCT <DCL LIST> ENDSTRUCT

<SYMBOLIC RANGE> ::= { <ID LIST> }

<TABLE TYPE> ::= TABLE <BOUNDS> OF <TYPE>

<THEN PART> ::= THEN <OPT STMT SEQ>

<TUPLE> ::= <OPT REPETITION FACTOR> { <OPT TUPLE BODY LIST> }
          | <OPT REPETITION FACTOR> <STRING>

<TUPLE BODY LIST> ::= <EXPRSN>
                    | <TUPLE BODY LIST> , <EXPRSN>

<TYPE> ::= <OPT PACKING> <OPT VAL> <UNPACKED TYPE>

<TYPE DCL STMT> ::= TYPE <DCL LIST>

<UNLABELLED STMT> ::= <ITERATIVE STMT>
                    | <IF STMT>
                    | <CASE STMT>
```

```
                    | <SELECT STMT>
                    | <EXIT STMT>
                    | <RETURN STMT>
                    | <BLOCK>
                    | <EXPRSN>


<UNPACKED TYPE> ::= <SIMPLE TYPE>
                  | <PROC TYPE>
                  | <AGGREGATE TYPE>

<QUALIFIED ID> ::= <ID>
                 | <ID> :: <ID>
<WITH DCL STMT> ::= WITH <STORAGE REF LIST>
```